Uwagi

2: KS: tytuł
9: ZL: Nie wiem, czy opisywać pozostałe operacje logiczne, tylko te dwie są wykorzystywane
9: KS: Na razie nie :-)
9: KS: `https://github.com/ZofiaLor/magister-granule`
12: KS: ¿Czy tutaj są dobre wartości?
12: KS: ¿na pewno?
13: ZL: liczebności takie, jak w tabelach, inne dla każdego zestawu pomiarów
13: KS: ¿na pewno?
13: ZL: dla małych danych (pierwsze 2) 4 razy, dla dużych – 1 raz

Contents

1 Introduction **2**

2 Related works **3**
2.1 Hierarchical clustering algorithms 3
2.2 Granular computing 6
2.2.1 Extensional fuzzy numbers 6

3 Granular agglomerative hierarchical clustering **9**

4 Experiments **11**
4.1 Datasets 12
4.2 Clustering time 12
4.3 Quality of clustering 13
4.4 Rozmyte grupowanie hierarchiczne 17

5 Badania **18**
5.1 Zbiory danych 18
5.2 Metodyka badań 19
5.2.1 Czas grupowania 20
5.2.2 Jakość grupowania 21
5.3 Wyniki 26
5.3.1 Czas grupowania 26
5.3.2 Jakość grupowania 27

6 Podsumowanie **48**

GrAgglo: A granular agglomerative clustering algorithm [tytuł]

17th October 2026

Abstract

Keywords: hierarchical clustering, fuzzy C-means, granular computing, fuzzy numbers, Python

1 Introduction

Clustering is a very common technique of the unsupervised data analysis. It has been widely used in many applications. Many techniques and algorithms have been proposed, implemented, and used.

The idea of data clustering is the partition of data into groups. Items in the same group are more similar to each other than to those in other groups. The similarity is usually defined by a distance function. Commonly the Euclidean distance is used, but other distance functions can be applied as well.

Multiple clustering techniques have been proposed in the literature. One of them is the hierarchical clustering. In the ascending hierarchical clustering, each data item is initially assigned to a separate cluster. Then, the closest clusters are merged into one cluster. This technique is not complicated, however it has a high time complexity, which is in general $O(n^3)$, where n is the number of data items. This may be a bottleneck in enormous data sets.

This problem can be addressed with granular computing. Granular computing (GrC) is a paradigm of data analysis with informative granules. The data are gathered in granules and the granules are an input of further algorithms. The representation of granules heavily depends on the application. Commonly set, fuzzy sets, rough sets, intervals and many other are used for granules.

Unfortunately, the mathematical representation of granules is not straightfoward for mathematical operations. Commonly, Zadeh's extension principle is used to elaborate the results. It causes three main problems [32]:

- for some fuzzy sets, interative numerical procedures are needed, the operations cannot be presented with closed-form formulae;
- for some fuzzy sets and operations, the fuzzy set in the result of an operation does not preserve the properties of the operands (the set of fuzzy numbers with operations is not complete);
- multiple arithmetical operations causes spread of fuzziness, the more operations run, the more fuzzy result, even beyond reasonable interpretability.

In this paper, we would like to address these two problems. We would like to use granular approach to accelerate hierarchical clustering algorithms with extensional fuzzy numbers.

2 Related works

2.1 Hierarchical clustering algorithms

Clustering is a well known technique for unsupervised data analysis. It aims at elaboration of a partition of data item in such a way that items in one cluster are more similar to items in the same cluster than to items in other clusters [11, 44].

Multiple paradigms are used for clustering algorithms [44], eg.:

- Hierarchical approach based on either merging small clusters into larger ones (ascending approach) or splitting larger clusters into smaller ones (descending approach) [44], eg. single-linkage [9, 35], SLINK [24], complete-linkage [37], gravitational clustering [42];
- Criterion function approach based on minimalisation of a certain criterion is represented by a huge FCM [3], family, eg. FCOM [15], FuBi [30], Rough FCM [27, 25], Subspace FCM [26] etc.
- Density approach locates clusters in the most dense regions of the input domain, eg. DBSCAN [8, 38], OPTICS [2].
- Graph approach treats data items as graph vertices and similarities as graph edges, eg. based on spectral clustering [18], deep methods [23], kernels [16], density [39], modularity [4], probability [40].

Hierarchical clustering algorithms are based on the idea of merging or splitting clusters in a hierarchical way. The ascending hierarchical clustering algorithm

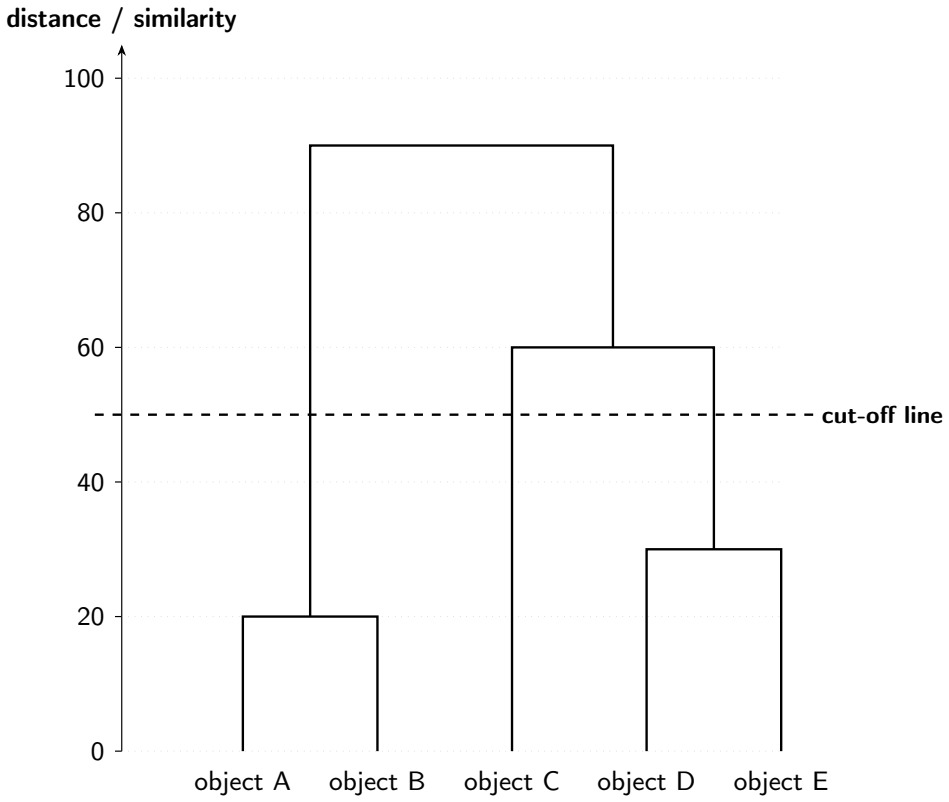


Figure 1: Example of a dendrogram for hierarchical clustering of five objects with a cut-off line at distance 50.

starts with each data item in a separate cluster. Then, the closest clusters are merged into one cluster. This process is repeated until all data items are in one cluster. Clustering can be visualized with a dendrogram (Fig. 1), which is a tree-like diagram that shows the hierarchical relationships between clusters. The height of the branches in the dendrogram represents the distance between clusters. A cut-off line can be drawn to determine the number of clusters. The distance between clusters can be defined in different ways, such as single-linkage, complete-linkage, average-linkage, or Ward's method. This choice affects the shape and size of the resulting clusters (Fig. 2). In the single-linkage approach [9, 35], the distance d between two clusters is defined as the minimum distance between any pair of points in the two clusters $\mathbb{A}$ and $\mathbb{B}$:

$$t\left(\mathbb{A}, \mathbb{B}\right) = \min_{\mathbf{a}\in\mathbb{A}, \mathbf{b}\in\mathbb{B}} t\left(\mathbf{a}, \mathbf{b}\right). \tag{1}$$

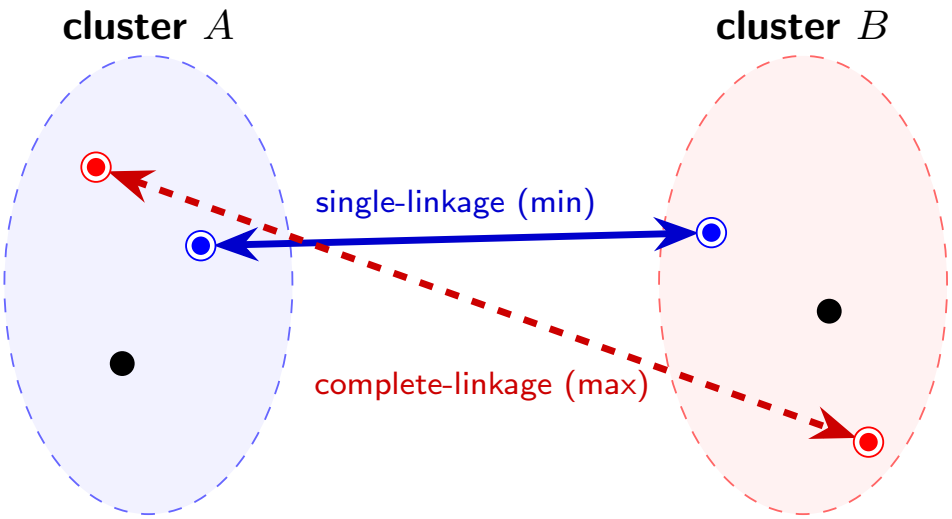


Figure 2: Example of single-linkage and complete-linkage methods for two clusters.

For complete-linkage [37, 17, 5], the distance between two clusters is defined as the maximum distance between any pair of points in the two clusters:

$$t\left(\mathbb{A},\mathbb{B}\right) = \max_{\mathbf{a}\in\mathbb{A},\mathbf{b}\in\mathbb{B}} t\left(\mathbf{a},\mathbf{b}\right). \tag{2}$$

In the complete-linkage approach when the distance between all data items do not exist, the dendrogram is not connected and has several not connected components.

Some more strategies are also used for defining the distance between clusters, such as WPGMA (Weighted Pair Group Method with Arithmetic Mean) [36], UPGMA (Unweighted Pair Group Method with Arithmetic Mean) [36], or the Ward's method [41].

This approach has a high time complexity. It is necessary to compute the distance between all pairs of clusters in each iteration, which results overall time complexity of $O(n^3)$. There are known implementations of single-linkage SLINK [24] and complete-linkage CLINK [5] algorithms with time complexity of $O(n^2)$. However, there are some limitations of these algorithms, eg. SLINK cannot handle a case when more pairs of clusters have the same minimal distance. In contrast, the FCM algorithm has a time complexity of $O(n)$, which is significantly lower.

2.2 Granular computing

Lofti Zadeh introduced a new paradigm of computation based on informative granules [49]. This is an abstraction that fills the gap between numbers and human concepts. Humans commonly do not operate on numbers in every day life. We simple operate on concepts that can be described with numbers. Zadeh named these concepts informative granules.

The main idea of granular computing (GrC) is to provide ideas, methods, algorithms, and techniques to make computer work in a way that is similar to human cognition [50]. The next step dreamt by Zadeh were computers calculating with words [51, 20].

Informative granule is a collection of similar items with a clear significance. This definition is very general. It does not define the similarity of the items nor the significance. These notions are specified in an application they are used for. Thus the representation of granules is very diverse [32], eg. intervals, sets, fuzzy sets, interval type-2 fuzzy sets [19], rough sets [34, 21], intuitionistic sets, shadowed sets, clusters [28], soft sets [43], if-then rules [29], extensional fuzzy numbers [32].

The next level of abstraction over informative granules is the triarchic theory of GrC based on three perspectives mutually intertwined: philosophical, methodological, and computational [47]. This is a base of the three-way decision approach [45, 46, 31, 22, 33].

One of representations of informative granules are extensional fuzzy numbers.

2.2.1 Extensional fuzzy numbers

Extensional fuzzy number is a number constructed with a extension of crisp numbers with a fuzzy $\star$-similarity relation defined for a left continuous t-norm $\star$ [12, 13, 14, 32]. This is an alternative to the Zadeh's extension principle [48] way for constructing fuzzy numbers. It has some advantages such as [32]:

- Extensional fuzzy numbers are complete with respect to the arithmetic operations: the result of an operation on extensional fuzzy numbers is a extensional fuzzy number.
- The infinite support of a fuzzy number is allowed.
- There is no spread of fuzziness: interative operations on extensional fuzzy numbers do not increase the vagueness of the result.
- The computational overload of extensional fuzzy numbers is very low: no discretisation, no integration, no iterations are needed.

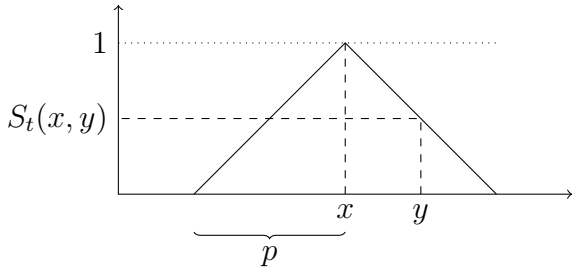


Figure 3: Fuzzy $\star$-similarity relation S_t [32].

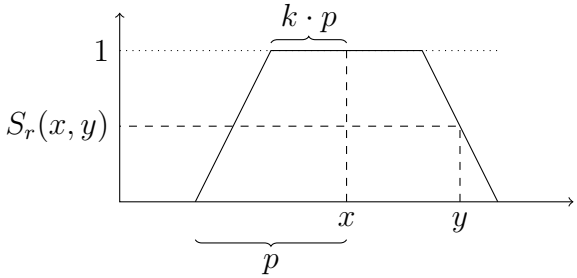


Figure 4: Fuzzy $\star$-similarity relation S_r [32].

- Infinite number of types of extensional fuzzy numbers can be easily constructed with various fuzzy $\star$-similarity relations.

Extensional fuzzy numbers can be constructed with any fuzzy $\star$-similarity relation that is defined as a left continuous t-norm that is reflexive, symmetric and satisfies the triangle property [32]. $\star$-similarity relations that correspond with common fuzzy sets are [32]:

- for symmetric triangular fuzzy sets (Fig. 3)

$$S_t(x, y) = \max\left\{1 - \frac{|x - y|}{p}, 0\right\}, \tag{3}$$

  where $p > 0$;

- for symmetric trapezoidal fuzzy sets (Fig. 4)

$$S_r(x, y) = \min\left\{1, \max\left\{1 - \frac{|x - y|}{p}, 0\right\} \cdot \frac{1}{1 - k}\right\}, \tag{4}$$

  where $p > 0$ and $0 \leqslant k < 1$;

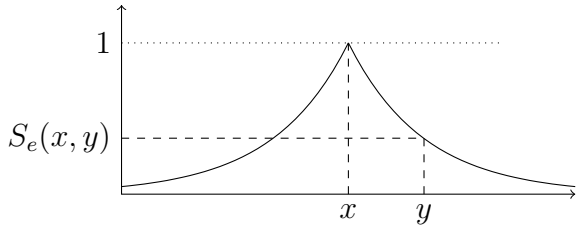


Figure 5: Fuzzy $\star$-similarity relation S_e for $p = 1$ [32].

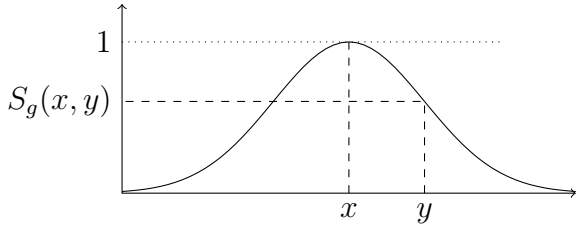


Figure 6: Fuzzy $\star$-similarity relation S_g for $p = 1$ [32].

- for exponential fuzzy sets (Fig. 5)

$$S_e\left(x, y\right) = \exp\left(-\frac{|x - y|}{p}\right), \tag{5}$$

  where $p > 0$;

- for Gaussian fuzzy sets (Fig. 6)

$$S_g\left(x, y\right) = \exp\left(-\frac{(x - y)^2}{2p^2}\right), \tag{6}$$

  where $p > 0$.

Having defined the $\star$-similarity relation the extensional fuzzy number x_S is defined as

$$\forall_{y \in \mathbb{R}} \qquad x_S(y) = S\left(x, y\right). \tag{7}$$

Thus x defines the core of the extensional fuzzy number and the parameter p is responsible for its uzziness. So $x_{S(p)}$ fully defines a extensional fuzzy number.

A one-argument arithmetical operation $\mathbf{op}$ on extensional fuzzy numbers based on a $\star$-similarity relation S is defined as [32]:

$$\mathbf{op}(x_S) = \mathbf{op}(x)_S, \tag{8}$$

for example

$$-\left(x_S\right) = \left(-x\right)_S \tag{9}$$
$$\left(x_S\right)^{-1} = \left(1/x\right)_S, \qquad \text{for } x_S \neq 0_S \tag{10}$$

A two-argument arithmetical operation **op** on extensional fuzzy numbers is defined as [32]:

$$x_{S(p_x)} \,\mathbf{op}\, y_{S(p_y)} = \left(x \,\mathbf{op}\, y\right)_{\max\{p_x,p_y\}}, \tag{11}$$

for example

$$x_{S(p_x)} + y_{S(p_y)} = \left(x + y\right)_{S(\max(p_x,p_y))} \tag{12}$$
$$x_{S(p_x)} \cdot y_{S(p_y)} = \left(x \cdot y\right)_{S(\max(p_x,p_y))} \tag{13}$$

The above formulae make the arithmetic with extensional fuzzy numbers very efficient computationally.

For extensional fuzzy numbers, the equality operator returns a fuzzy value from the interval $[0, 1]$ that is defined as a $\star$-similarity relation S:

$$\left(x_{S(p_x)} =_S y_{S(p_y)}\right) = S_{(\max\{p_x,p_y\})}(x, y). \tag{14}$$

The fuzzy operator $<_S$ is defined as:

$$\left(x_{S(p_x)} <_S y_{S(p_y)}\right) = \begin{cases} 1 - \left(x_{S(p_x)} =_S y_{S(p_y)}\right) & : x \leq y \\ 0 & : x > y \end{cases}. \tag{15}$$

[Nie wiem, czy opisywać pozostałe operacje logiczne, tylko te dwie są wykorzystywane] [Na razie nie :-)]

3 Granular agglomerative hierarchical clustering

The hierarchical clustering algorithms have in general a high time complexity – in the case of the general algorithm up to $O(X^3)$, where X is the number of data points. This problem can be mitigated with granular computation. Instead of clustering the original data, first the data is granulated, and then the hierarchical clustering is applied to the granules. In our approach, the granules are represented as extensional fuzzy numbers. [https://github.com/ZofiaLor/magister-granule]

In the first step (line 2 in Alg. 1), the original data $\mathbb{X}$ is clustered with the fuzzy c-means algorithm FCM. FCM returns the matrix of centres $\mathbf{V}$ of G clusters and

Algorithm 1 GRAGGLO – granular agglomerative hierarchical clustering

Require: $\mathbb{X}$: input data
Require: G: number of granules
Require: C: expected number of clusters
Require: r: type of the $\star$-relation
Require: ξ: threshold for the fuzzy minimum operation

procedure GRAGGLO(zX, G, r, ξ)
  $\mathbf{V}, \mathbf{U} \leftarrow$ FCM($\mathbb{X}, G$) ▷ clusterisation of the input data, $\mathbf{V}$: centres of clusters, $\mathbf{U}$: memberships of data items to clusters, $O(XG)$
  $\mathbb{G} \leftarrow$ CLUSTERSTOGRANULES($\mathbf{V}, \mathbf{U}, r$) ▷ $O(XG)$
  for each granule g_1 in $\mathbb{G}$ **do**
    for each granule $g_2 \neq g_1$ in $\mathbb{G}$ **do**
      $\mathbf{D}[g_1, g_2] \leftarrow$ FUZZYDISTANCE(g_1, g_2)
    end for
  end for
  $n \leftarrow G$ ▷ each granule is a separate cluster
  while $n > C$ **do** ▷ $O(G)$
    $i, j \leftarrow$ FINDINDICESOFTHECLOSESTGRANULES($\mathbf{D}, \mathbf{p}, r, \xi$) ▷ $O(G^2)$
    $\mathbf{r} \leftarrow []$ ▷ for the merged cluster, create a new row $\mathbf{r}$...
    $\mathbf{c} \leftarrow []$ ▷ ... and a new column $\mathbf{c}$ for the distance matrix $\mathbf{D}$
    for all granules g in $\mathbb{G}$ **do** ▷ merge the found clusters into one
      $\mathbf{r}[g] = \mathbf{c}[g] =$ FUZZYAGGREGATION($\mathbf{D}[i, g], \mathbf{D}[j, g], r, \xi$)
    end for
    remove the i-th and j-th rows and columns from the matrix $\mathbf{D}$
    add the new row $\mathbf{r}$ and column $\mathbf{c}$ to the distance matrix $\mathbf{D}$
    $n \leftarrow n - 1$ ▷ update the number of clusters
  end while
end procedure

the matrix $\mathbf{U}$ of memberships of data points to clusters. The obtained results are then transformed into granules with the CLUSTERSTOGRANULES procedure (line 3 in Alg. 1). Granules are represented with extensional fuzzy numbers, which are defined by the following parameters: the centre of the granule v_{cd}, which is equal to the centre of the cluster obtained from FCM, and the fuzziness p_{cd}, which is calculated based on the memberships $\mathbf{U}$ of data points to clusters with the following formula:

$$p_{cd} = \sqrt{\frac{\sum_{i=1}^{X} \mu_{ci}^{m} (x_{id} - v_{cd})^2}{\sum_{i=1}^{X} \mu_{ci}^{m}}}. \tag{16}$$

Then (lines 4–8) two nested loops are responsible for elaboration of fuzzy distance between granules. The distances are stored in the distance matrix $\mathbf{D}$. The crucial part of the algorithm is the while loop (lines 10–20), which implements the agglomerative hierarchical clustering of granules. Each iteration of the loop merges the two closest granules into one, and updates the distance matrix $\mathbf{D}$ according to the selected type of aggregation method in the FUZZYAGGREGATION procedure (line 15). In the granular single-linkage method, the fuzzy minimum operation is used, while in the granular complete-linkage method, the fuzzy maximum operation is used. Thus, each iteration reduces the number of granules by one (line 19), until the expected number of clusters C is reached.

The granulation of the input data $\mathbb{X}$ with the FCM algorithm and tranformation of clusters into granules has a time complexity of $O(XG)$, where X is the number of data points and G number of granules. The most time-consuming part of the algorithm is the while loop, which has to be executed $G - C$ times. In each iteration, the algorithm has to find the two closest granules in the distance matrix $\mathbf{D}$, which has a size of $G \times G$. This makes the time complexity of the while loop $O(G^3)$. Finally, the overall time complexity of the GRAGGLO algorithm is $O(XG + G^3)$, which is significantly lower than the time complexity of the general agglomerative hierarchical clustering algorithm $O(X^3)$, because $G \ll X$.

4 Experiments

The experiments aim at evaluating the performance of the proposed granular agglomerative hierarchical clustering algorithm GRAGGLO in comparison to the general agglomerative hierarchical clustering algorithm in the perspective of the time complexity and the quality of clustering results.

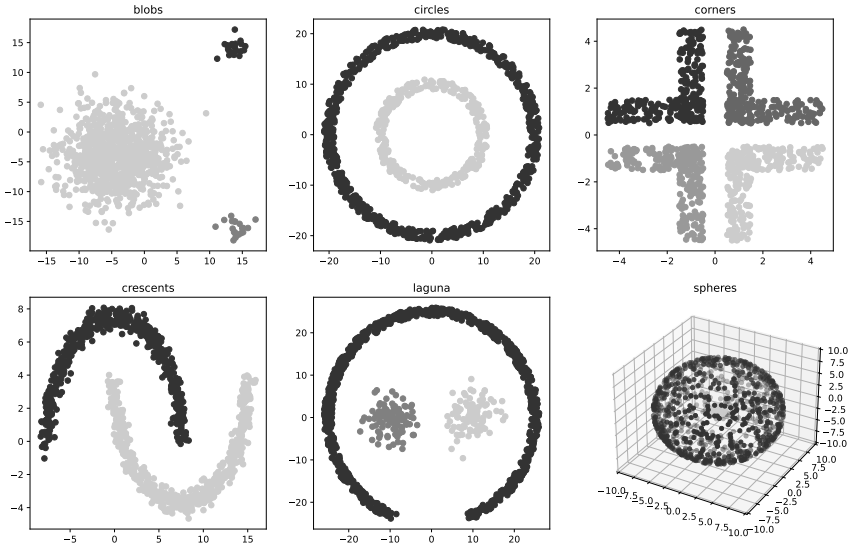


Figure 7: Visualization of synthetic datasets of size 1000. Points of the same color belong to the same target cluster.

4.1 Datasets

Two groups of datasets are used in the experiments. The first one is synthetic labelled data in six shapes ('blobs', 'circles', 'corners', 'crescents', 'laguna'and 'spheres') and eight sizes (1000, 2000, 5000, 10000, 20000, 30000, 40000 and 50000 elements [¿Czy tutaj są dobre wartości?]. The data is visualized in Fig. 7. The second type are the real-life unlabelled 2D data 'brightkite'[1] (with $4\,491\,143$ data items) and 'gowalla'[2] (with $6\,442\,890$ data items). In these two datesets, only the latitude and longitude columns are used for the experiments. The data is visualized in Fig. 8.

4.2 Clustering time

The clustering time is measured for both the general agglomerative hierarchical clustering algorithm and the proposed granular agglomerative hierarchical clustering algorithm GRAGGLO. The experiments is run for various number of input data points $X \in \{500, 1000, 1500, \ldots, 10000\}$ [¿na pewno?] [liczebności

[1] `https://snap.stanford.edu/data/loc-brightkite.html`
[2] `https://snap.stanford.edu/data/loc-gowalla.html`

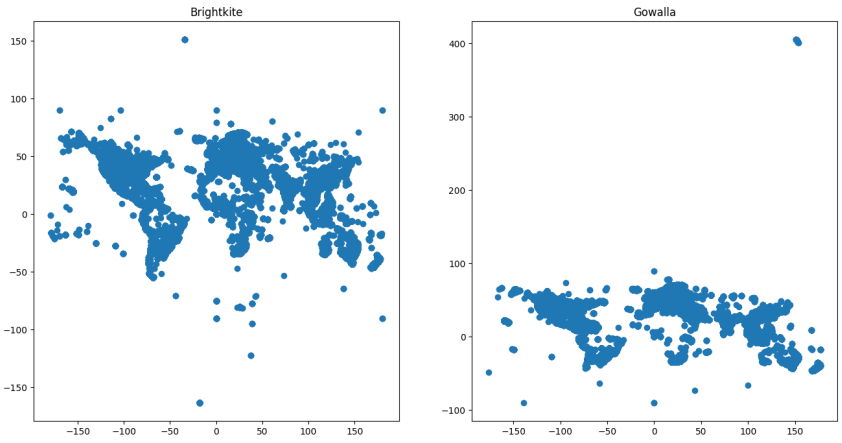


Figure 8: Visualization of 'brightkite' and 'gowalla' datasets.

takie, jak w tabelach, inne dla każdego zestawu pomiarów] and various number of granules $G \in \{50, 100, 200\}$. Each experiment is repeated 10 [¿na pewno?] [dla małych danych (pierwsze 2) 4 razy, dla dużych – 1 raz] times and the average time is reported.

All experiments confirm the linear time complexity of GRAGGLO algorithm. Figure 10 compares its execution time to `scikit-learn`'s `AgglomerativeClustering` algorithm (single-linkage). For more than 40000 data points, GRAGGLO algorithm is faster than `AgglomerativeClustering` even with 200 granules.

However, FCM's memory complexity must be taken into account: the algorithm allocates a membership array of floating-point numbers with size $G \times X$. Figure 11 shows that for a combination of large numbers of data points and granules, the execution time may increase significantly (200 granules, above 3 million data points case) or FCM might fail to allocate the array due to lack of memory (100 and 200 granules, 6 million data points case).

4.3 Quality of clustering

Table 1: Comparison of the average execution times (in [ms]) of the agglomerative clustering and the GRAGGLO for different data sizes and granule numbers.

	agglomerative	GRAGGLO, number of granules		
data size	clustering	50	100	200
500	256.15	79.31	166.93	527.55
1000	1092.4	131.34	300.99	944.47
1500	2725.91	189.25	564.14	1269.19
2000	7125.25	280.46	664.66	1546.45
2500	14676.58	331.97	821.36	1848.87
3000	24289.29	474.25	1000.07	2197.36
3500	36747.48	554.55	1122.44	2546.56
4000	52439.32	629.62	1310.48	2901.85
4500	71895.08	696.23	1460.06	3233.01
5000	95497.87	778.06	1590.62	3540.35

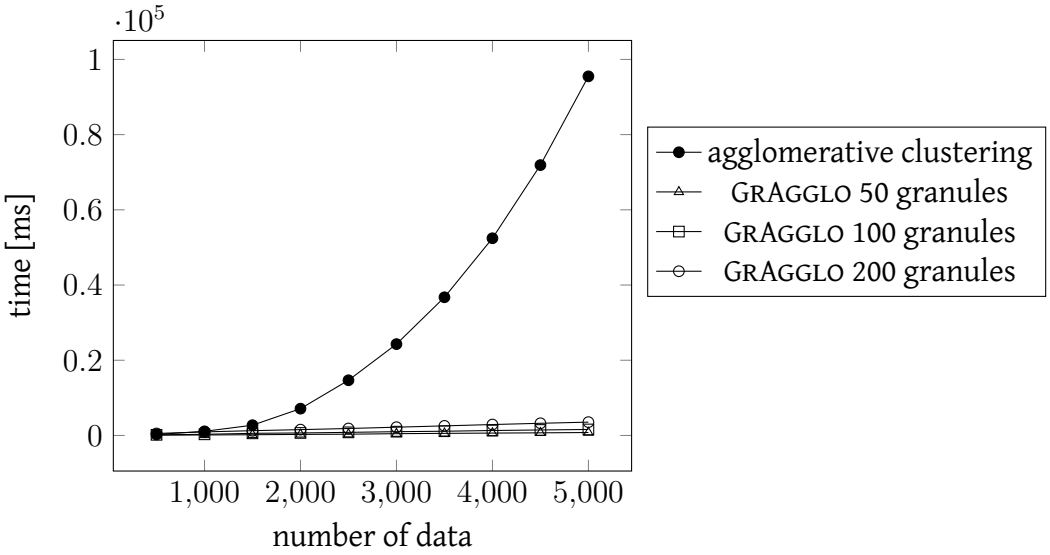


Figure 9: Comparison of the average execution times (in [ms]) of the agglomerative clustering and the GRAGGLO for different data sizes and granule numbers.

Table 2: Comparison of average execution times (in [ms]) of the `scikit-learn` library's strict and proposed fuzzy agglomerative hierarchical clustering algorithms for different data and granule numbers.

	agglomerative	GRAGGLO, number of granules		
data size	clustering	50	100	200
1000	27.68	140.37	278.82	922.37
2000	39.44	252.88	662.75	1488.29
5000	312.70	774.75	1535.61	3344.13
10000	1425.18	1465.33	3017.63	6566.77
20000	6434.67	2925.11	6101.59	13076.91
30000	15171.27	4659.39	9518.74	20115.51
40000	29034.08	6226.38	12677.87	26848.27
50000	47815.40	7811.42	15775.42	33288.67

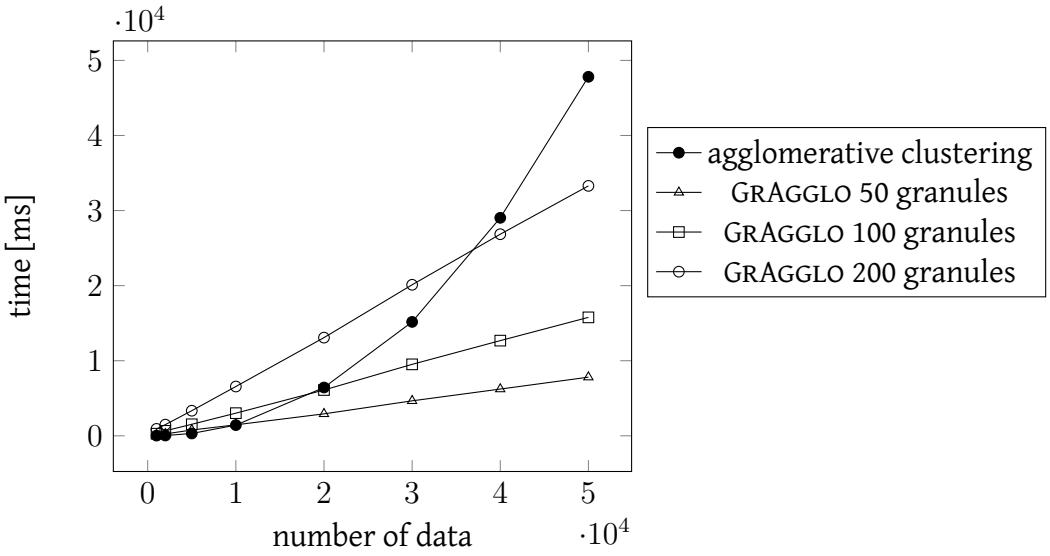


Figure 10: Comparison of average execution times of the `scikit-learn` library's strict and proposed fuzzy agglomerative hierarchical clustering algorithms for different data and granule numbers.

Table 3: Comparison of execution times (in [s]) of fuzzy agglomerative hierarchical clustering algorithm GRAGGLO for different data and granule numbers, for large data numbers.

	number of granules		
data size	50	100	200
50000	7.72	15.84	33.23
100000	15.53	31.72	67.03
500000	79.73	177.07	348.11
1000000	158.69	322.65	675.32
2000000	300.08	641.60	1348.01
3000000	468.20	967.35	2126.28
4000000	628.20	1328.02	5400.40
5000000	799.02	1696.94	8650.32
6000000	964.80	–	–

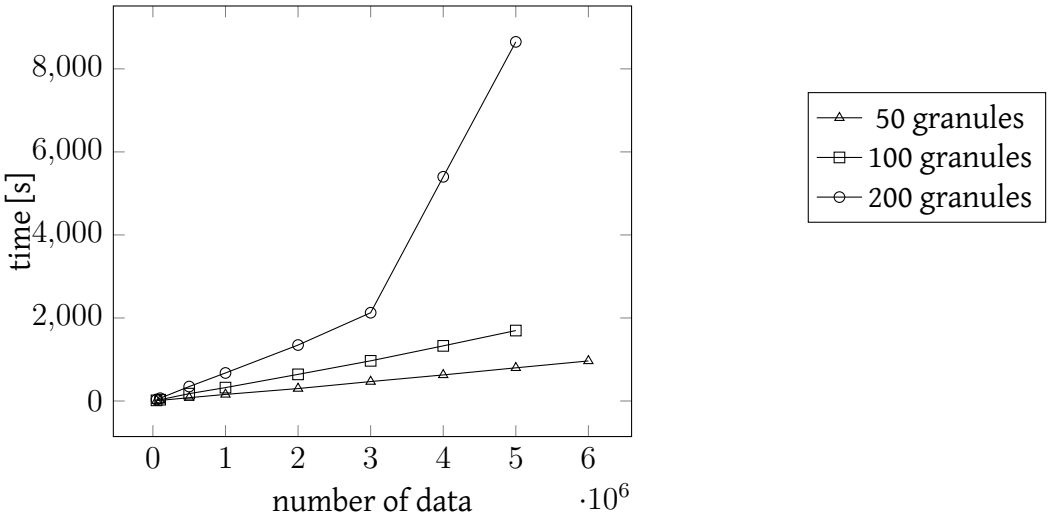


Figure 11: Comparison of execution times of fuzzy agglomerative hierarchical clustering algorithm GRAGGLO for different data and granule numbers, for large data numbers.

Algorithm 2 Minimum rozmyte

Require: a, b: porównywane rozszerzalne liczby rozmyte
Require: r: typ relacji (trójkątna, wykładnicza, gaussowska)
Require: ξ: wartość progowa
1: **procedure** FUZZYMINIMUM(a, b, r, ξ)
2: **if** $(a <_{S_r} b) > \xi$ **then**
3: **return** a
4: **else**
5: **return** b
6: **end if**
7: **end procedure**

4.4 Rozmyte grupowanie hierarchiczne

Zbiór granul $\mathbb{G}$ oraz ich kształt są parametrami wejściowymi algorytmu rozmytego aglomeracyjnego grupowania hierarchicznego FUZZYAGGLO. Na początku wyznaczane są rozmyte odległości euklidesowe (wzór (**??**))między każdą parą granul, przy pomocy operacji arytmetycznych określonych w sekcji **??** (alg. 5). Wynikiem obliczeń jest symetryczna macierz odległości $\mathbf{D}$ o wymiarach $X \times X$, gdzie $d_{ij} = d_{ji} = d(\mathbf{x}_i, \mathbf{x}_j)$. Odległość danego elementu od samego siebie (d_{ii}) nie jest wykorzystywana w algorytmie, jej wartość nie jest więc istotna.

Grupowanie hierarchiczne FUZZYAGGLO ma postać zmodyfikowanego algorytmu naiwnego PRIMITIVECLUSTERING, o złożoności czasowej $O(G^3)$. Zmieniony musi zostać sposób wyznaczania kolejnych minimalnych odległości w macierzy $\mathbf{D}$ oraz, zależne od łączności, aktualizacje odległości do nowo scalonego klastra.

Minimum i maksimum wyznacza się przy pomocy operacji porównania. W przypadku rozszerzalnych liczb rozmytych, wynikami takich operacji są liczby z przedziału $[0, 1]$. W celu określenia, która z liczb rozmytych jest większa lub mniejsza, wprowadzony zostaje parametr progowy ξ, będący parametrem badawczym. Algorytmy rozmytego minimum i maksimum przedstawiają odpowiednio pseudokody 2 i 3. Na podstawie alg. 2 można też utworzyć rozmytą funkcję $\arg\min$ (alg. 4).

Dla łączności SINGLELINKAGE, algorytm FUZZYAGGLO ma postać opisaną pseudokodem **??**. Ten sam algorytm dla łączności COMPLETELINKAGE jest niemal identyczny: należy jedynie podmienić wywołanie algorytmu rozmytego minimum (alg. 2) w linii 15 na algorytm rozmytego maksimum (alg. 3).

Algorithm 3 Maksimum rozmyte

Require: a, b: porównywane rozszerzalne liczby rozmyte
Require: r: typ relacji (trójkątna, wykładnicza, gaussowska)
Require: ξ: wartość progowa
1: **procedure** FUZZYMAXIMUM(a, b, r, ξ)
2: **if** $(a >_{S_r} b) > \xi$ **then**
3: **return** a
4: **else**
5: **return** b
6: **end if**
7: **end procedure**

5 Badania

Działanie algorytmu GRAGGLO musi zostać zbadane pod kątem czasu wykonania oraz jakości grupowania, również w porównaniu z jego ostrym odpowiednikiem.

5.1 Zbiory danych

Pierwszym zbiorem danych są dane syntetyczne, które znajdują się w plikach `.data` zlokalizowanych w folderze `dane`, pogrupowanych w podfoldery według kształtów. Dla każdego kształtu zbiory mają liczebność 1000, 2000, 5000, $10\,000$, $20\,000$, $30\,000$, $40\,000$ oraz $50\,000$ wierszy.

Dane mają postać współrzędnych punktów w dwóch lub trzech wymiarach – każdy wiersz w pliku reprezentuje współrzędne jednego punktu, współrzędne oddzielone są znakami tabulacji. Taka liczba wymiarów pozwala na wizualizację całości danych w postaci wykresu. W zależności od kształtu, dla danego zbioru można na podstawie wykresu wyróżnić 2, 3 lub 4 wyraźnie odseparowane klastry. Wykresy danych przedstawiono na rys. 7, a w tabeli 4 – dodatkowo ich charakterystykę (liczbę wymiarów i klastrów).

Dane te nie posiadają etykiet. W sposób opisany w sekcji na stronie 22 każdemu punktowi przypisano etykietę w postaci liczby całkowitej $[0, C-1]$ i zapisano w folderze `dane_labelled` w analogicznym formacie.

Wykorzystano również zbiory danych 'brightkite' [6] ($4\,491\,143$ wierszy) oraz 'gowalla' [7] ($6\,442\,890$ wierszy). Dane przedstawiają informacje o logowaniach się użytkowników platform społecznościowych. Dane przycięto, aby zawierały jedynie kolumny „latitude" oraz „longitude" – dwuwymiarowe współrzędne lokalizacji logowań. Dane nie są etykietowane i nie mają przypisanych klas. Analiza wykresów (rys. 8) pozwala zauważyć kształty kontynentów – można uznać je

Algorithm 4 Rozmyta funkcja $\arg\min$

Require: $\mathbf{X}$: macierz liczb rozmytych, $\mathbf{p}$: wektor indeksów, dla których znajdowane jest minimum, r: typ relacji (trójkątna, wykładnicza, gaussowska), ξ: parametr porównania

1: **procedure** FUZZYARGMIN($\mathbf{X}, r, \xi$)
2: result $\leftarrow \infty_{S(0)}$ ▷ liczba rozmyta, dla której $x = \infty, p = 0$
3: $(w, k) \leftarrow (0, 0)$ ▷ Współrzędne (wiersz, kolumna) wartości minimalnej w przeszukiwanej części macierzy $\mathbf{X}$
4: **for** $i = 1$ to $|\mathbf{p}| - 1$ **do**
5: **for** $j = i + 1$ to $|\mathbf{p}|$ **do**
6: $p_w \leftarrow \mathbf{p}[i]$
7: $p_k \leftarrow \mathbf{p}[j]$
8: **if** ($\mathbf{X}[p_w][p_k] <_{S_r}$ result) > ξ **then**
9: result $\leftarrow \mathbf{X}[p_w][p_k]$
10: $(w, k) \leftarrow (p_w, p_k)$
11: **end if**
12: **end for**
13: **end for**
14: **return** (w, k) ▷ Współrzędne (wiersz, kolumna) wartości minimalnej w przeszukiwanej części macierzy $\mathbf{X}$
15: **end procedure**

Figure 12: Wykresy danych o różnych kształtach z folderu `dane` o rozmiarze 1000 punktów.

za docelowe klastry. Dane zawierają błędne współrzędne. W danych 'brightkite', kolumna „latitude" (szerokość geograficzna) zawiera wartości z przedziału $[-163.193\,333, 151.198]$, podczas gdy dopuszczalnym przedziałem jest $[-90, 90]$. Dla danych 'gowalla' przedział ten wynosi $[-90, 405.658\,498]$. Błędne współrzędne pozostawiono w zbiorze danych.

5.2 Metodyka badań

Algorytm GRAGGLO (**??**) zbadano pod kątem szybkości wykonania oraz jakości grupowania poprzez porównanie jego efektów z efektami ostrego algorytmu aglomeracyjnego grupowania hierarchicznego SINGLELINKAGE i COMPLETELINKAGE.

Figure 13: Wykresy danych 'brightkite'i 'gowalla'.

Algorithm 5 Rozmyte odległości euklidesowe

Require: $\mathbf{X}$: macierz liczb rozmytych
1: **procedure** FUZZYDISTANCE($\mathbf{X}$)
2: result $\leftarrow \infty_{S(0)}$ ▷ liczba rozmyta, dla której $x = \infty, p = 0$
3: $X \leftarrow |\mathbb{X}|$ ▷ liczba danych
4: $\mathbf{D} \leftarrow$ macierz zerowa $X \times X$
5: **for** $i = 1$ to $X - 1$ **do**
6: **for** $j = i + 1$ to X **do**
7: $d_{ij} \leftarrow 0_0$ ▷ liczba rozmyta, dla której $x = 0, p = 0$
8: **for** k in $|\mathbf{X}[i]|$ **do** ▷ dla każdego wymiaru liczby rozmytej
9: $d_{ij} \leftarrow d_{ij} + (\mathbf{X}[i][k] - \mathbf{X}[j][k])^2$ ▷ Operacje arytmetyczne na liczbach rozmytych (wzór **(??)**)
10: **end for**
11: $d_{ij} \leftarrow \sqrt{d_{ij}}$ ▷ Operacja arytmetyczna na liczbie rozmytej (wzór **(??)**)
12: $\mathbf{D}[i][j] \leftarrow \mathbf{D}[j][i] \leftarrow d_{ij}$ ▷ macierz jest symetryczna względem przekątnej
13: **end for**
14: **end for**
15: **return D** ▷ macierz rozmytych odległości między parami liczb rozmytych
16: **end procedure**

5.2.1 Czas grupowania

Aby znaleźć charakterystykę zależności czasu wykonania algorytmów od liczby danych, przygotowano dane o rozmiarach 500-$10\,000$ z krokiem 500. W tym celu brane były początkowe podzbiory ze zbiorów danych o rozmiarze $10\,000$. Jako że nieistotne były wyniki grupowania, nie było konieczne losowe próbkowanie.

Przyjęto też stałą maksymalną liczbę 80 iteracji algorytmu rozmytych c-średnich. Sekcja 5.3.2 pokazuje, że jest to wystarczająca wartość. Złożoność algorytmu jest liniowa w zależności od liczby iteracji [10]. Algorytm wykonano też dla różnych liczności granul: 50, 100 oraz 200.

Table 4: Charakterystyka zbiorów danych dołączonych do projektu.

	'blobs'	'circles'	'corners'	'crescents'	'laguna'	'spheres'
liczba klastrów	3	2	4	2	3	2
liczba wymiarów	2	2	2	2	2	3

Czas pojedynczego wykonania algorytmu ostrego zmierzono poprzez różnicę czasu rozpoczęcia i zakończenia grupowania hierarchicznego. Wykorzystany został algorytm `AgglomerativeClustering` biblioteki `scikit-learn` (łączność SINGLELINKAGE) oraz własna implementacja ostrego algorytmu grupowania, analogiczna do wersji rozmytej. Pozwoliło to na lepszy pogląd na wpływ granulacji na szybkość, jako że sama logika grupowania we własnej implementacji pozostała ta sama, co w algorytmie rozmytym. Algorytm biblioteczny ma z kolei najniższą złożoność czasową dla łączności SINGLELINKAGE (wykorzystuje algorytm minimalnego drzewa rozpinającego), można więc porównać czas wykonania GRAGGLO z najlepszym możliwym czasem grupowania tych samych danych algorytmem ostrym. Do tego porównania wykorzystano oryginalne dane wejściowe, we wszystkich rozmiarach (1000-$50\,000$). W czasie pojedynczego wykonania algorytmu rozmytego zawarto czas wykonania algorytmu rozmytych c-średnich, utworzenia na jego podstawie listy granul oraz rozmytego grupowania hierarchicznego.

Poszczególne pomiary czasu działania algorytmu dla dokładnie takich samych rozmiarów danych mogą różnić się o kilka milisekund. Wynika to między innymi z działania w tle innych procesów komputera, na którym przeprowadzane są badania. Typowym rozwiązaniem tego problemu jest kilkukrotne powtórzenie pomiarów dla tych samych danych i obliczenie średniej arytmetycznej wyników. Podczas badań pomiary powtórzono czterokrotnie, co wystarczyło do wyeliminowania odstających wyników.

Wykonano również pomiary czasu wykonania algorytmu GRAGGLO dla dużych liczebności danych: $50\,000$, $100\,000$, $500\,000$, $1\,000\,000$, $2\,000\,000$, $3\,000\,000$, $4\,000\,000$, $5\,000\,000$ i $6\,000\,000$, dla liczebności granul 50, 100 i 200. Jako że dla tak dużych rozmiarów danych czas ten jest rzędu minut, nie mają znaczenia milisekundowe przekłamania wyników. Z tego powodu wystarczyło wykonać jeden pomiar dla każdego rozmiaru danych.

5.2.2 Jakość grupowania

Jakość zmierzono poprzez porównanie wyników algorytmu GRAGGLO z oczekiwanymi przynależnościami punktów do klastrów, oznaczonymi etykietami. Jako miary jakości wykorzystano dokładność, trafność oraz precyzję.

Pomiary wykonano dla wszystkich kształtów i rozmiarów danych. W przypadku algorytmu rozmytego, pomiary przeprowadzono również dla różnych łączności (SINGLELINKAGE, COMPLETELINKAGE), liczebności granul (50, 100, 200), typu relacji liczb rozmytych (trójkątne, wykładnicze, gaussowskie) oraz parametru $\xi \in \{0.05, 0.10, 0.15, \ldots, 0.95\}$

Podobnie jak w przypadku pomiarów czasu grupowania, przyjęto stałą maksymalną liczbę 80 iteracji. Sparametryzowanie tej wartości oznaczałoby badanie działania bibliotecznej funkcji FCM, zamiast własnego algorytmu.

Kolejność danych została przed pomiarem losowo zmieszana przy pomocy

generatora liczb pseudolosowych o ustalonym ziarnie. Dzięki temu, przy każdym uruchomieniu kolejność ta jest identyczna, wystarczyły więc jednokrotne pomiary wyników działania algorytmu GRAGGLO dla każdej kombinacji parametrów.

Etykietowanie danych W celu przypisania etykiet do danych z folderu `dane`, przygotowano pomocniczy skrypt w pliku `label_assignment.py`. W zależności od tego, czy dane są dwu- czy trójwymiarowe, przypisanie etykiet odbywa się na inny sposób.

Dane dwuwymiarowe Oznaczanie etykiet dla danych dwuwymiarowych wykorzystuje widżet `LassoSelector` biblioteki `matplotlib`. Dla każdego pliku z danymi dwuwymiarowymi, wyświetlono wykres połączony z obiektem klasy `SelectFromCollection` z przykładu [1]. Klasa ta między innymi umożliwia wyznaczenie indeksów i współrzędnych punktów zaznaczonych myszą na wykresie.

Każdemu punktowi domyślnie przypisano etykietę 0. Po zaznaczeniu zbioru punktów na wykresie naciśnięcie klawisza cyfry na klawiaturze powoduje przypisanie tym punktom etykiety o danym numerze i wyświetlenie w konsoli informacji, ilu punktom ją przypisano. Proces można powtarzać dowolną liczbę razy, można go zakończyć poprzez naciśnięcie klawisza ENTER i zamknięcie wykresu.

W celu ostatecznej weryfikacji pojawia się drugi wykres, na którym każdej etykiecie odpowiada inny kolor. Po zamknięciu tego wykresu, plik z etykietami jest zapisywany w folderze `dane_labelled`. Plik ma format zgodny z oryginalnym plikiem z danymi (każdy wiersz to oddzielone znakami tabulacji współrzędne jednego punktu), z dodaną dodatkową kolumną z etykietami punktów.

Dane trójwymiarowe W przypadku danych trójwymiarowych skorzystano z faktu, że jedyne dane wykorzystywane w pracy, dla których należy znaleźć etykiety, to dane 'spheres'. Punkty otrzymane z tych danych tworzą dwie sfery: mniejsza, wewnętrzna sfera jest zamknięta w sześcianie o wymiarach $1 \times 1 \times 1$ jednostek, a punkty drugiej, większej sfery znajdują się poza tym sześcianem. Wszystkim punktom, których wszystkie współrzędne mają wartości mniejsze lub równe 1 przypisano etykietę 1, pozostałym – 0. Dane z etykietami zostały zapisane w folderze `dane_labelled`, w takim samym formacie, jak dane dwuwymiarowe.

Wyniki etykietowania Na wykresie 14 przedstawiono wyniki etykietowania danych. Punkty oznaczone tym samym kolorem mają te same etykiety.

W przypadku danych 'laguna' o liczebności $20\,000$ i większej, dwa środkowe klastry przestają być wyraźnie oddzielone od siebie (rys. 15). Liczebności tych klastrów po podziale ich narzędziem `LassoSelector` nie są równe, jak w przypadku mniejszych zbiorów o tym samym kształcie – różnice wynoszą nie więcej niż 18

Figure 14: Wykresy danych o różnych kształtach z folderu `dane_labelled` o rozmiarze 1000 punktów, o etykietach oznaczonych kolorami.

Figure 15: Wykres danych 'laguna' z folderu `dane_labelled` o rozmiarze $20\,000$ punktów, o etykietach oznaczonych kolorami.

punktów. Przy tak dużych zbiorach danych wpływ pojedynczych punktów jest marginalny, należy jednak uwzględnić tę niepewność w analizie wyników jakości grupowania.

Miara jakości Wybranymi miarami jakości grupowania są dokładność A, trafność R i precyzja P.

W przypadku dwuklasowym (klasy positive i negative), dokładność (ang. accuracy) jest definiowana jako:

$$A = \frac{TP + TN}{TP + FP + TN + FN}, \tag{17}$$

gdzie:

- TP – liczba danych poprawnie zaklasyfikowanych do klasy positive
- FP – liczba danych niepoprawnie zaklasyfikowanych do klasy positive
- TN – liczba danych poprawnie zaklasyfikowanych do klasy negative
- FN – liczba danych niepoprawnie zaklasyfikowanych do klasy negative

Dokładność jest łatwo rozszerzalna na więcej klas: sumę liczb danych poprawnie zaklasyfikowanych do każdej klasy należy podzielić przez liczbę wszystkich danych.

Trafność (czułość, ang. recall) definiowana jest jako:

$$R = \frac{TP}{TP + FN}, \tag{18}$$

czyli stosunek liczby danych poprawnie zaklasyfikowanych do klasy positive do wszystkich danych do niej należących.

Precyzję (ang. precision) definiuje się jako:

$$P = \frac{TP}{TP + FP}, \tag{19}$$

czyli stosunek liczby danych poprawnie zaklasyfikowanych do klasy positive do wszystkich danych które zostały do niej zaklasyfikowane.

Trafność i precyzję można rozszerzyć na wiele klas na różne sposoby. W badaniach wybrano metodę, w której najpierw obliczana jest wartość metryk cząstkowych dla każdej klasy z osobna (jedna klasa odpowiada klasie positive, wszystkie pozostałe – negative), a następnie obliczana jest z nich średnia arytmetyczna. Oznacza to, że każda klasa ma jednakową wagę dla ostatecznego wyniku – jeżeli będą miały różne liczności, to klasy zawierające małą liczbę danych będą mieć proporcjonalnie większy wpływ na wyniki trafności i precyzji, niż dokładności.

Przynależność punktów do granul Rozmyty algorytm grupowania jako wynik zwraca przynależność do klastrów granul, nie oryginalnych danych. Na potrzeby porównania konieczne jest określenie, do której granuli należy każdy punkt. Przynależność tą wyznaczano na dwa sposoby na podstawie (1) macierzy przynależności do klastra wyznaczonego przez algorytm rozmytych c-średnich $\boldsymbol{\mu}$ i (2) wartości relacji $*$-podobieństwa S.

Pierwsze podejście dla każdego punktu i, znajduje granulę, dla której wartość odpowiedniego wiersza w macierzy przynależności $\boldsymbol{\mu}_i$ jest maksymalna.

Drugie podejście przedstawia pseudokod 6. Dla każdego punktu i, przynależność do granuli j obliczana na podstawie wartości operacji $=_S$ (wzór **??**) dla każdego wymiaru punktu i granuli. Operacja ta jest zależna od kształtu granul, dla liczb trójkątnych, wykładniczych i gaussowskich należy wykorzystać odpowiednio wzory **??**, **??** lub **??** uwzględniając, że dla liczby ostrej (współrzędnej oryginalnego punktu) $p = 0$.

Wyniki dla każdej współrzędnej łączone są t-normą algebraiczną (wzór **??**). Wreszcie określa się, że punkt należy do tej granuli, dla której wynik tego złączenia jest maksymalny.

Należy uwzględnić, że rozszerzalne liczby rozmyte trójkątne mają skończony nośnik. Oznacza to, że punkt może nie należeć do żadnej granuli – dla każdej z nich przynależność będzie zerowa. Punkt taki uznawany jest za szum i nie bierze udziału w wyznaczaniu wartości dokładności, trafności i precyzji, jednak odsetek punktów stanowiących szum jest w tym przypadku dodatkową miarą jakości.

Rysunek 16 przedstawia przykład takiego przypisania punktów do granul, dla czytelności wykresów wyznaczono jedynie 10 granul. Wykresy dla liczb wykładniczych i gaussowskich są bardzo podobne, ponadto wszystkie punkty są przypisane do granuli. W przypadku liczb trójkątnych, duża część punktów stanowi szum.

(a) Liczby trójkątne (b) Liczby wykładnicze (c) Liczby gaussowskie

Figure 16: Przynależność punktów do granul w zależności od kształtu liczb rozmytych. 10 środków granul oznaczono kolorem czerwonym, punkty należące do różnych granul oznaczone są różnymi kolorami. Kolorem szarym oznaczono punkty stanowiące szum.

Macierz pomyłek i obliczanie miar jakości Wyznaczono macierz $\mathbf{L}$, w której wartość elementu l_{ij} jest równa liczbie punktów, dla których oczekiwana etykieta ma wartość i i przynależących do klastra j wyznaczonego przez algorytm GRAGGLO:

$$l_{ij} = \sum_{k=0}^{N} [x_k \in i \wedge x_k \in j]. \tag{20}$$

Sposób wyznaczenia tej macierzy przedstawiono również na pseudokodzie 7.

Numeracja klastrów wyznaczonych przez algorytm GRAGGLO może być inna od oczekiwanych etykiet ze względu na różną kolejność ich wyznaczenia. Wykorzystując algorytm przedstawiony na pseudokodzie 8, wyznaczono taką permutację wierszy i kolumn macierzy $\mathbf{L}$, która maksymalizuje wartości na przekątnej.

Z uzyskanej w ten sposób macierzy pomyłek, dokładność wyznaczana jest jako suma wartości na przekątnej macierzy podzielona przez całkowitą liczbę danych:

$$A = \frac{\sum_{i=0}^{C} l_{ii}}{N} \tag{21}$$

lub

$$A = \frac{\sum_{i=0}^{C} l_{ii}}{\sum_{i=0}^{C} \sum_{j=0}^{C} l_{ij}}, \tag{22}$$

gdzie C to liczba klastrów.

Trafność (R) i precyzja (P) wyznaczane są najpierw osobno dla każdego klastra c:

$$R_c = \frac{l_{cc}}{\sum_{i=0}^{C} l_{ci}}, \tag{23}$$

$$P_c = \frac{l_{cc}}{\sum_{i=0}^{C} l_{ic}}. \tag{24}$$

Trafność i precyzja dla całości wyników otrzymywane są jako średnia arytmetyczna metryk częściowych:

$$R = \frac{\sum_{i=0}^{C} R_i}{C}, \tag{25}$$

$$P = \frac{\sum_{i=0}^{C} P_i}{C}. \tag{26}$$

Sposób obliczenia metryk jakości przedstawia również pseudokod 9.

Jeżeli dane zawierają szum, może on być interpretowany na dwa sposoby w kontekście wyznaczania metryk:

1. Szum nie jest brany pod uwagę, analizowane są jedynie dane niebędące szumem.

2. Szum uznawany jest za błędną klasyfikację. Aby skorygować wartość metryki o szum, należy pomnożyć ją przez odsetek punktów niebędących szumem w stosunku do całości danych. Na przykład, jeżeli początkowa wartość metryki wynosi 80%, a 40% danych stanowi szum, to ostateczna wartość tej metryki będzie równa $80\% * (100\% - 40\%) = 48\%$.

Oprócz niezależnych procentowych średnich wartości metryk dokładności, trafności i precyzji istotne jest, jakie są wyniki działania GRAGGLO w porównaniu z algorytmem ostrym. W tym celu wykorzystano algorytm `AgglomerativeClustering` biblioteki `scikit-learn` aby zgrupować dane oryginalne, a także środki klastrów (w liczebnościach 50, 100, 200) wyznaczonych przez algorytm rozmytych c-średnich, bez uwzględnienia rozmycia, czyli macierzy $\boldsymbol{\mu}$.

Do wyznaczenia metryk jakości dla tych badań, również wykorzystano algorytmy 7, 8 oraz 9. W przypadku oryginalnych danych, wejściem algorytmu 7 były etykiety wyznaczone przez algorytm `AgglomerativeClustering`. Dla środków klastrów algorytmu rozmytych c-średnich, najpierw wyznaczono przynależności punktów do tych klastrów na podstawie macierzy $\boldsymbol{\mu}$ (tak samo jak dla GRAGGLO).

5.3 Wyniki

5.3.1 Czas grupowania

Algorytm GRAGGLO, który wykorzystuje wstępną granulację FCM, ma znacząco mniejszy czas wykonania w stosunku do własnej implementacji algorytmu ostrego. Charakterystykę czasu wykonania algorytmów w zależności od liczby danych przedstawiono na rys. 17.

Figure 17: Charakterystyka czasu wykonania własnych implementacji algorytmów ostrego i rozmytego dla różnej liczby granul od liczby danych wejściowych.

Figure 18: Charakterystyka czasu wykonania algorytmu ostrego `AgglomerativeClustering` biblioteki `scikit-learn` i własnej implementacji algorytmu rozmytego dla różnej liczby granul od liczby danych wejściowych.

Dla obydwu algorytmów, złożoność czasowa grupowania hierarchicznego jest sześcienna i wynosi dla algorytmu ostrego $O(X^3)$, dla rozmytego $O(G^3)$, gdzie liczba ziaren G jest stała i zdecydowanie mniejsza od liczby danych ($G \ll X$). Złożoność czasowa algorytmu rozmytych c-średnich wynosi $O(X)$, więc jest on mniej wrażliwy na zwiększanie rozmiaru danych wejściowych.

Porównano również czas wykonania algorytmu GRAGGLO z algorytmem `AgglomerativeClustering` biblioteki `scikit-learn`, dla złożoności SINGLELINKAGE. Wyniki przedstawiono na rys. 18.

Wykres pokazuje, że dla małych danych (do $10\,000$ punktów), najkrócej wykonuje się algorytm biblioteczny – klasteryzacja algorytmem rozmytych c-średnich trwa stosunkowo długo, mimo złożoności liniowej. Powyżej $10\,000$ danych algorytm GRAGGLO dla 50 granul wykonywany jest szybciej niż algorytm ostry. Od $40\,000$ danych, jest on szybszy dla wszystkich badanych liczebności granul. Oznacza to, że mimo gorszej złożoności samego algorytmu hierarchicznego FUZZYAGGLO, całość algorytmu rozmytego jest lepsza pod względem czasu wykonania od algorytmu `AgglomerativeClustering`, powyżej pewnej liczności danych.

Czasy wykonania algorytmu rozmytego GRAGGLO na dużych danych przedstawia rys. 19. W przypadku 200 granul zauważyć można zaburzenie liniowości wyników. Spowodowane jest zbyt dużym wykorzystaniem pamięci: sama macierz $\boldsymbol{\mu}$ ma wymiary $X \times G$, czyli dla 4 milionów danych wejściowych zawiera ona 800 milionów liczb zmiennoprzecinkowych. Z tego samego powodu możliwe było wykonanie algorytmu na 6 milionach danych tylko dla 50 granul – dla większej liczby nie wystarczyło pamięci.

5.3.2 Jakość grupowania

Na wykresach wyników grupowania hierarchicznego metodą rozmytą punkty szare odpowiadają współrzędnym oryginalnych danych. Wyjątkiem są wykresy

Figure 19: Charakterystyka czasu wykonania algorytmu rozmytego dla różnej liczby granul od liczby danych wejściowych dla dużych danych.

Table 5: Dokładność grupowania hierarchicznego danych algorytmem ostrym o łączności SINGLELINKAGE

	dokładność			
kształt	dane oryginalne	50 granul	100 granul	200 granul
'blobs'	0.9899	0.9999	0.9939	0.9696
'circles'	1.0000	1.0000	1.0000	1.0000
'corners'	1.0000	1.0000	1.0000	1.0000
'crescents'	1.0000	1.0000	1.0000	1.0000
'laguna'	0.9125	0.9997	0.9998	0.9999
'spheres'	1.0000	1.0000	1.0000	1.0000

danych 'spheres', gdzie oryginalne punkty pominięto, aby zachować czytelność wykresu. Punkty kolorowe odpowiadają środkom granul, gdzie środki o tym samym kolorze należą do tego samego klastra. Elipsy (elipsoidy dla 'spheres') w kolorze różowym wizualizują rozmycie granul, gdzie długość każdej półosi jest równa dwukrotności rozmycia w danym wymiarze. Mnożnik ten został wybrany ze względu na dobrą czytelność wynikowych wykresów.

Wszystkie wykresy zamieszczono w katalogu `img/granule_visualization`, pogrupowane w podkatalogach według kształtu danych ('blobs', 'circles', 'corners', 'crescents', 'laguna' i 'spheres'). Nazwa każdego pliku ma format `[kształt]-[liczba_danych][relacja]_[łączność].pdf`, gdzie „relacja": `t` (trójkątna), `e` (wykładnicza) lub `g` (gaussowska), a „łączność": `s` (SINGLELINKAGE) lub `c` (COMPLETELINKAGE).

Dla niektórych wyników utworzono również wykresy na podstawie sumarycznej przynależności punktów do granul z danego klastra. Mają postać kilku podwykresów, gdzie punktami w kolorze niebieskim oznaczono dane oryginalne. Dla każdego podwykresu opisanego jako „Klaster c", dla każdego punktu x wyznaczana jest suma $\sum_{g=0}^{G} \mu_{xg} \cdot [g \in c]$, gdzie zapis $[P]$ wartościowany jest do 1, gdy warunek P jest spełniony, w przeciwnym razie do 0 (wartość μ_{xg} jest wliczana do sumy, gdy granula g należy do klastra c). Wyższe wartości tej sumy oznaczają większą przynależność punktu do tego klastra. Im wyższa wartość sumy, tym ciemniejszy kolor punktu na wykresie.

Dane syntetyczne Wyniki dokładności, trafności i precyzji otrzymane na podstawie ostrego algorytmu hierarchicznego `AgglomerativeClustering` grupującego dane oryginalne i środki klastrów wyznaczonych przez algorytm rozmytych c-średnich przedstawiają odpowienio tabele 5, 6 i 7 dla łączności SINGLELINKAGE oraz 8, 9 i 10 dla łączności COMPLETELINKAGE.

Table 6: Trafność grupowania hierarchicznego danych algorytmem ostrym o łączności SINGLELINKAGE

	trafność			
kształt	dane oryginalne	50 granul	100 granul	200 granul
'blobs'	0.8333	1.0000	0.9571	0.7487
'circles'	1.0000	1.0000	1.0000	1.0000
'corners'	1.0000	1.0000	1.0000	1.0000
'crescents'	1.0000	1.0000	1.0000	1.0000
'laguna'	0.7084	0.9990	0.9995	0.9997
'spheres'	1.0000	1.0000	1.0000	1.0000

Table 7: Precyzja grupowania hierarchicznego danych algorytmem ostrym o łączności SINGLELINKAGE

	precyzja			
kształt	dane oryginalne	50 granul	100 granul	200 granul
'blobs'	0.8295	0.9990	0.9392	0.7282
'circles'	1.0000	1.0000	1.0000	1.0000
'corners'	1.0000	1.0000	1.0000	1.0000
'crescents'	1.0000	1.0000	1.0000	1.0000
'laguna'	0.8375	0.9996	0.9998	0.9998
'spheres'	1.0000	1.0000	1.0000	1.0000

Table 8: Dokładność grupowania hierarchicznego danych algorytmem ostrym o łączności COMPLETELINKAGE

	dokładność			
kształt	dane oryginalne	50 granul	100 granul	200 granul
'blobs'	0.7261	0.9999	0.8956	0.7237
'circles'	0.5620	0.5533	0.5330	0.5627
'corners'	0.5265	0.4818	0.5053	0.5130
'crescents'	0.7986	0.8012	0.8032	0.7822
'laguna'	0.4508	0.4875	0.4653	0.4664
'spheres'	0.5852	0.5712	0.5650	0.6615

Table 9: Trafność grupowania hierarchicznego danych algorytmem ostrym o łączności CompleteLinkage

	trafność			
kształt	dane oryginalne	50 granul	100 granul	200 granul
'blobs'	0.7825	1.0000	0.8822	0.9041
'circles'	0.4900	0.4940	0.4876	0.5071
'corners'	0.5265	0.4818	0.5053	0.5130
'crescents'	0.7986	0.8012	0.8033	0.7822
'laguna'	0.5447	0.6026	0.4856	0.5072
'spheres'	0.5473	0.3729	0.4805	0.5897

Table 10: Precyzja grupowania hierarchicznego danych algorytmem ostrym o łączności CompleteLinkage

	precyzja			
kształt	dane oryginalne	50 granul	100 granul	200 granul
'blobs'	0.6054	0.9990	0.8753	0.6591
'circles'	0.4881	0.4948	0.4880	0.5052
'corners'	0.4956	0.4779	0.4631	0.4828
'crescents'	0.8372	0.8072	0.8285	0.8093
'laguna'	0.3984	0.4302	0.3561	0.3804
'spheres'	0.5143	0.4480	0.4891	0.5324

Table 11: Dokładność grupowania hierarchicznego danych w zależności od ich kształtu i wykorzystanej łączności

	dokładność	
kształt	SINGLELINKAGE	COMPLETELINKAGE
'blobs'	0.9874	0.8758
'circles'	0.9982	0.5738
'corners'	0.9485	0.5167
'crescents'	0.9943	0.7977
'laguna'	0.9876	0.4685
'spheres'	0.9876	0.7056

Table 12: Trafność grupowania hierarchicznego danych w zależności od ich kształtu i wykorzystanej łączności

	trafność	
kształt	SINGLELINKAGE	COMPLETELINKAGE
'blobs'	0.8846	0.8968
'circles'	0.9971	0.4970
'corners'	0.9485	0.5167
'crescents'	0.9943	0.7977
'laguna'	0.9625	0.5571
'spheres'	0.9512	0.4086

Kształt danych Średnią dokładność grupowania danych w zależności od ich kształtu dla łączności SINGLELINKAGE i COMPLETELINKAGE przedstawiono w tabeli 11, trafność – w tabeli 12, a precyzję – w tabeli 13.

W przypadku SINGLELINKAGE, najlepszą średnią dokładność, trafność i precyzję zaobserwowano dla danych 'circles'. Dane 'corners' mają najgorszą dokładność, a dane 'blobs' – najgorszą trafność i precyzję.

Co istotne, dla większości plików wejściowych, za wyjątkiem niektórych danych 'blobs' i 'laguna', maksymalne wartości wszystkich metryk jakości wynosi 100%. Oznacza to, że istnieje przynajmniej jedna taka kombinacja parametrów (typu relacji, liczby granul, parametru ξ), dla których grupowanie przebiega w pełni zgodnie z oczekiwaniami.

'blobs' Wykresy wyników grupowania danych 'blobs' (rys. 20) pokazują, że wysoka dokładność uzyskana dla nich z macierzy pomyłek nie przekłada się na

Table 13: Precyzja grupowania hierarchicznego danych w zależności od ich kształtu i wykorzystanej łączności

	precyzja	
kształt	SINGLELINKAGE	COMPLETELINKAGE
'blobs'	0.8727	0.8325
'circles'	0.9984	0.4947
'corners'	0.9523	0.4840
'crescents'	0.9964	0.8207
'laguna'	0.9812	0.4003
'spheres'	0.9480	0.4442

zgodność wyników z oczekiwaniami. Ze względu na dużą dysproporcję danych pomiędzy trzema klastrami ($48 : 1 : 1$), algorytm rozmytych c-średnich często traktuje dwa mniejsze z nich jako szum i nie tworzy dla nich granul. Zawierają one jednak jedynie 4% danych, dlatego dokładność grupowania pozostaje wysoka. Trafność i precyzja lepiej odzwierciedlają zgodność z oczekiwaniami – są znacznie niższe od dokładności. Mniejsze klastry mają tę samą wagę, co większy klaster, dlatego niepoprawne grupowania wyraźnie wpływają na wartość metryki. Jako że błędy wynikają z klasteryzacji FCM, wyniki uzyskane dla grupowania samych środków granul są bardzo zbliżone do uzyskanych przy pomocy GRAGGLO.

Jak pokazuje tabela 14, dla niektórych liczebności danych nie istnieje kombinacja parametrów, dla której wartości metryk jakości wynoszą 100%. W tych przypadkach granule wyznaczane są w poprawnych miejscach, natomiast pojedyncze punkty oryginalnych danych zostają przypisane do niepoprawnej granuli. Dla tych liczebności, wartości metryk otrzymane przez algorytm ostry również nie osiągają 100%.

'circles' Dane 'circles' (rys. 21) mają najwyższe wyniki dokładności, trafności i precyzji. Dla mniejszej liczby ($G = 50$) rozmyte odległości między granulami położonymi na zewnętrznym okręgu są większe niż między okręgiem zewnętrznym a wewnętrznym, jedyne błędy grupowania pojawiają się dla tej liczebności granul, dla wysokich wartości parametru ξ ($\xi \in \{0.9, 0.95\}$). W znacznej większości przypadków, dokładność grupowania wynosi 100%, tak jak dla grupowania ostrego. Okrąg zewnętrzny jest też bardziej liczny niż wewnętrzny (proporcja 7:3). Dysproporcja ta nie jest jednak tak duża, jak w przypadku 'blobs', algorytm rozmytych c-średnich tworzy granule w równomiernych odstępach.

Table 14: Maksymalne wartości metryk jakości dla danych 'blobs' przy łączności SINGLELINKAGE.

	jakość		
rozmiar danych	dokładność	trafność	precyzja
1000	1.0000	1.0000	1.0000
2000	1.0000	1.0000	1.0000
5000	0.9998	0.9999	0.9967
10 000	1.0000	1.0000	1.0000
20 000	1.0000	1.0000	1.0000
30 000	0.9999	1.0000	0.9989
40 000	1.0000	1.0000	1.0000
50 000	0.9998	1.0000	0.9997

'corners' Klastry dla danych 'corners' mają nierównomierną gęstość, przez co algorytm rozmytych c-średnich tworzy więcej granul bliżej środka układu współrzędnych (rys. 22). Odległości między granulami położonymi na „ramionach" i w „środkach" klastrów są na tyle duże, że rozmyte odległości „środków" różnych klastrów mogą być od nich mniejsze. Algorytm GRAGGLO osiąga więc średnio gorsze wyniki niż algorytm ostry, choć wiele kombinacji parametrów pozwala na uzyskanie 100-procentowej dokładności. Wszystkie klastry mają tę samą liczebność, wartości dokładności, trafności i precyzji są zbliżone do siebie.

'crescents' Dane 'crescents' tworzą dwa klastry o identycznej liczebności i gęstości, dzięki czemu dokładność, trafność i precyzja ich grupowania są prawie najlepsze (rys. 23). Ze względu na bliskie położenie końców łuków, błędy wciąż mogą się pojawić – koniec jednego łuku złączony zostaje z łukiem drugim. Dzieje się tak tylko dla bardzo wysokich wartości parametru ξ – 0.9 i 0.95.

'laguna' Dla danych 'laguna' błędy mogą pojawić się, gdy pojedyncze granule utworzone zostają w większej odległości od środkowego klastra, do którego oczekiwane jest, że powinny należeć. Punkt taki może wtedy zostać zakwalifikowany, jako osobny klaster, co na rysunku 24 pokazuje przykład dla 50 granul i $\xi = 0.9$. Podobny błąd przedstawia wykres dla 200 granul i $\xi = 0.9$ – pewna grupa punktów na otaczającym okręgu jest na tyle oddalona od pozostałych, że zostaje uznana za osobny klaster, jeden ze środkowych klastrów jest łączony z pozostałą częścią okręgu.

Powyżej liczebności $20\,000$ punktów włącznie, dokładność nigdy nie osiąga 100% (tab. 15), jednak najczęściej wynosi powyżej 99.9%. Jak zaznaczono w sekcji

Table 15: Maksymalne wartości metryk jakości dla danych ‘laguna’ przy łączności SINGLELINKAGE.

	jakość		
rozmiar danych	dokładność	trafność	precyzja
1000	1.0000	1.0000	1.0000
2000	1.0000	1.0000	1.0000
5000	1.0000	1.0000	1.0000
10 000	1.0000	1.0000	1.0000
20 000	0.9999	0.9997	0.9998
30 000	0.9999	0.9998	0.9999
40 000	0.9999	0.9998	0.9998
50 000	0.9997	0.9990	0.9997

dotyczącej etykietowania danych (sekcja na stronie 22), środkowe klastry są na tyle zbliżone do siebie, że na podstawie wykresu nie można definitywnie określić, które punkty pomiędzy nimi należą do którego z nich. Pojedyncze punkty mogą więc być klasyfikowane niepoprawnie z punktu widzenia przypisanych do nich etykiet.

Ilustrują to wykresy 25 oraz 26. Na rys. 25 punktów jest jedynie 1000 i wszystkie klastry są wyraźnie oddzielone od siebie. Każdy punkt wyraźnie należy do granul jednego klastra. Na rys. 26 punktów jest $40\,000$. Klastry środkowe (o numerach 1 i 2) pokrywają się w pobliżu współrzędnych (0, 0). Punkty w bliskim otoczeniu tych współrzędnych przynależą więc w podobnym stopniu do klastra 1 i klastra 2. Ponadto punkty położone na zewnątrz klastrów 1 i 2 są położone blisko otaczającego je okręgu, oznaczonego jako klaster 0. Można zauważyć ich wysoki stopień przynależności do tego klastra.

Dzięki temu nałożeniu się klastrów, jakość grupowania tych danych jest lepsza, gdy wykorzystany zostanie algorytm GRAGGLO, niż gdy zgrupowane zostaną dane oryginalne. Bez wstępnej klasteryzacji algorytmem rozmytych c-średnich, środkowe klastry są łączone ze sobą.

Proporcja danych wynosi 8:1:1 (otaczający okrąg ma większą liczebność), jednak jakość grupowania jest na tyle wysoka, że zarówno dokładność, jak i trafność i precyzja mają podobne wartości.

‘spheres’ W przypadku danych ‘spheres’ (rys. 27) można zauważyć, że wyniki poprawiają się wraz ze zwiększeniem liczby granul i zmniejszeniem wartości parametru ξ. Proporcja liczebności klastrów wynosi 9:1, trafność i precyzja są o około 4 punkty procentowe mniejsze od dokładności.

Figure 20: Efekty wykonania grupowania GRAGGLO dla danych 'blobs' o rozmiarze 1000 i wykładniczej relacji między liczbami rozmytymi, przy łączności SINGLELINKAGE.

Figure 21: Efekty wykonania grupowania GRAGGLO dla danych 'circles' o rozmiarze $50\,000$ i euklidesowej relacji między liczbami rozmytymi, przy łączności SINGLELINKAGE.

Figure 22: Efekty wykonania grupowania GRAGGLO dla danych 'corners' o rozmiarze 1000 i gaussowskiej relacji między liczbami rozmytymi, przy łączności SINGLELINKAGE.

Figure 23: Efekty wykonania grupowania GRAGGLO dla danych 'crescents' o rozmiarze 5000 i wykładniczej relacji między liczbami rozmytymi, przy łączności SINGLELINKAGE.

Figure 24: Efekty wykonania grupowania GRAGGLO dla danych 'laguna' o rozmiarze 1000 i wykładniczej relacji między liczbami rozmytymi, przy łączności SINGLELINKAGE.

Figure 25: Przynależność punktów do klastrów dla danych 'laguna' o rozmiarze 1000.

Figure 26: Przynależność punktów do klastrów dla danych 'laguna' o rozmiarze $40\,000$.

Figure 27: Efekty wykonania grupowania hierarchicznego granul dla danych 'spheres' o rozmiarze 5000 i gaussowskiej relacji między liczbami rozmytymi, przy łączności SINGLELINKAGE.

Figure 28: Efekty wykonania grupowania GRAGGLO dla danych 'blobs' o rozmiarze 1000 i trójkątnej relacji między liczbami rozmytymi, przy łączności COMPLETELINKAGE.

Figure 29: Efekty wykonania grupowania GRAGGLO dla danych 'circles' o rozmiarze 1000 i gaussowskiej relacji między liczbami rozmytymi, przy łączności COMPLETELINKAGE.

COMPLETELINKAGE Najlepsze wyniki dla łączności COMPLETELINKAGE uzyskano dla danych 'blobs'. Inaczej jest w przypadku najgorszych wyników: dane 'laguna' mają najgorszą dokładność i precyzję, a dane 'spheres' – najgorszą trafność. Wszystkie wyniki są znacznie gorsze od uzyskanych dla SINGLELINKAGE, za wyjątkiem trafności dla danych 'blobs'. Jak widać na wykresach danych 'blobs' (rys. 28), podobnie jak dla łączności SINGLELINKAGE, jeżeli granule zostaną wyznaczone przez algorytm rozmytych c-średnich we wszystkich trzech docelowych klastrach, to grupowanie FUZZYAGGLO przebiega poprawnie. Z kolei jeżeli granule utworzone zostaną tylko w największym z klastrów, to wyznaczone zostają w nim trzy klastry o podobnych liczebnościach. W takich przypadkach dokładność, trafność i precyzja grupowania będą niższe, niż dla SINGLELINKAGE, gdzie prawie wszystkie granule przypisywane są do tego samego klastra.

Mimo różnic w wartościach miar jakości, wnioski dla grupowania danych 'circles' (rys. 29), 'corners' (rys. 30), 'crescents' (rys. 31), 'laguna' (rys. 32) i 'spheres' (rys. 33) są podobne. Algorytm COMPLETELINKAGE, jak opisano w podsekcji **??**, dobrze znajduje gęste, kuliste klastry. Wymienione dane mają inne kształty, widać, że znalezione przez algorytm klastry są w przybliżeniu owalne lub elipsoidalne. Kształt danych wydaje się nie być brany pod uwagę przez algorytm.

Wyniki te należy też analizować w porównaniu z wynikami algorytmu ostrego (tabele 8, 9 i 10). Dla kształtów danych 'blobs' oraz 'spheres' nastąpiła znacząca poprawa dokładności, trafności i precyzji. Dla pozostałych danych wyniki są zbliżone do wyników algorytmu ostrego.

Przynależność określona na podstawie relacji $*$-podobieństwa Wyniki dla pomiarów, gdzie przynależność punktów do granul określono na podstawie relacji $*$-podobieństwa przedstawiono w tabelach 16, 17 i 18. Dodatkowo, gdy

Figure 30: Efekty wykonania grupowania GRAGGLO dla danych 'corners' o rozmiarze 1000 i wykładniczej relacji między liczbami rozmytymi, przy łączności COMPLETELINKAGE.

Figure 31: Efekty wykonania grupowania GRAGGLO dla danych ‘crescents’ o rozmiarze 1000 i trójkątnej relacji między liczbami rozmytymi, przy łączności COMPLETELINKAGE.

Figure 32: Efekty wykonania grupowania GRAGGLO dla danych ‘laguna’ o rozmiarze 1000 i gaussowskiej relacji między liczbami rozmytymi, przy łączności COMPLETELINKAGE.

szum nie jest uwzględniany, dla każdego pliku wejściowego istnieje taka kombinacja parametrów, dla której wartości metryk wynoszą 100%. Dla relacji trójkątnej, tylko pewien podzbiór punktów najbliżej środków granul będzie uwzględniony, jest więc mniejsza szansa na niezgodne z oczekiwaniami przypisanie do granuli. Jak pokazuje rys. 16, wiąże się to jednak z utratą dużego odsetka informacji.

Gdy szum nie jest uwzględniany w obliczaniu metryk, ich wartości są zbliżone do metryk dla przynależności określonej na podstawie macierzy $\boldsymbol{\mu}$ (tabele 11, 12 i 13), natomiast gdy jest on uwzględniony, wyniki są niższe o nawet kilkanaście punktów procentowych.

Podsumowanie Podobnie jak w przypadku algorytmu ostrego, GRAGGLO osiąga znacznie lepsze wyniki jakości grupowania wykorzystując łączność SINGLELINKAGE niż COMPLETELINKAGE. W większości przypadków, algorytm rozmyty jest w stanie osiągnąć te same, a nawet lepsze wyniki niż algorytm ostry.

Poprawa w szczególności zachodzi w sytuacji, gdy oczekiwane klastry zaczynają się pokrywać i istnieją punkty bliskie różnym grupom, na przykład dla danych ‘laguna’ o dużych liczebnościach. Dla algorytmu ostrego, punkt taki może spowodować złączenie różnych klastrów, co pokazano na rys. **??**. W przypadku algorytmu rozmytego granule są uogólnieniem grupy danych, co eliminuje takie pojedyncze punkty. Ponadto algorytm rozmytych c-średnich wyznacza granule bliżej środków klastrów, co dodatkowo pomaga w poprawnym grupowaniu.

Z drugiej strony, algorytm FCM może wprowadzić błędy w fazie przygotowania danych, jak pokazał przypadek danych ‘blobs’. Algorytm może błędnie wyznaczyć granule, gdy dane są bardzo niezrównoważone pod względem liczebności klastrów.

Figure 33: Efekty wykonania grupowania hierarchicznego granul dla danych ‘spheres’ o rozmiarze 5000 i gaussowskiej relacji między liczbami rozmytymi, przy łączności COMPLETELINKAGE.

Table 16: Dokładność grupowania hierarchicznego danych w zależności od ich kształtu i wykorzystanej łączności, gdy przynależność określana jest na podstawie relacji $*$-podobieństwa

	dokładność			
	bez szumu		z szumem	
kształt	SINGLELINKAGE	COMPLETELINKAGE	SINGLELINKAGE	COMPLETELINKAGE
'blobs'	0.9900	0.8755	0.8639	0.7609
'circles'	0.9982	0.5596	0.8136	0.4623
'corners'	0.9488	0.5248	0.7700	0.4285
'crescents'	0.9943	0.7869	0.8188	0.6522
'laguna'	0.9870	0.4748	0.7995	0.3827
'spheres'	0.9871	0.6665	0.7806	0.5485

Table 17: Trafność grupowania hierarchicznego danych w zależności od ich kształtu i wykorzystanej łączności, gdy przynależność określana jest na podstawie relacji $*$-podobieństwa

	trafność			
	bez szumu		z szumem	
kształt	SINGLELINKAGE	COMPLETELINKAGE	SINGLELINKAGE	COMPLETELINKAGE
'blobs'	0.8585	0.8508	0.7474	0.7435
'circles'	0.9971	0.4976	0.8125	0.4056
'corners'	0.9485	0.5229	0.7699	0.4277
'crescents'	0.9943	0.7868	0.8188	0.6521
'laguna'	0.9629	0.5566	0.7769	0.4513
'spheres'	0.9512	0.4083	0.7450	0.3251

Table 18: Precyzja grupowania hierarchicznego danych w zależności od ich kształtu i wykorzystanej łączności, gdy przynależność określana jest na podstawie relacji $*$-podobieństwa

	precyzja			
	bez szumu		z szumem	
kształt	SINGLELINKAGE	COMPLETELINKAGE	SINGLELINKAGE	COMPLETELINKAGE
'blobs'	0.8525	0.8278	0.7412	0.7179
'circles'	0.9984	0.4949	0.8138	0.4034
'corners'	0.9524	0.4894	0.7740	0.4006
'crescents'	0.9962	0.8105	0.8207	0.6718
'laguna'	0.9768	0.4077	0.7920	0.3277
'spheres'	0.9477	0.4151	0.7418	0.3411

Figure 34: Przynależność punktów do klastrów dla danych 'blobs' o rozmiarze 1000.

Liczba ziaren Średnią dokładność, trafność i precyzję grupowania danych w zależności od liczby granul przedstawiono odpowiednio w tabelach 19, 20 i 21. W przypadku łączności SINGLELINKAGE najlepsze wyniki osiągane są dla 100 granul, a dla COMPLETELINKAGE – dla 50.

Dla danych takich, jak 'circles' (rys. 21) czy 'crescents' (rys. 23), gdzie klastry mają podobną liczebność i gęstość, zwiększenie liczby granul powoduje ich zagęszczenie, przy równomiernym rozłożeniu.

Inaczej jest w przypadku danych 'blobs' (rys. 20). Dla większych liczebności granul, możliwe jest, że algorytm rozmytych c-średnich nie wyznaczy ani jednego środka granuli w dwóch mniejszych klastrach, przez co niemożliwe jest poprawne grupowanie. Tak samo dzieje się, gdy pogrupowane zostaną same środki granul, bez uwzględnienia rozmycia. Jak można odczytać z rys. 34, w takim przypadku większość punktów zostanie zaklasyfikowane do tego samego klastra.

Podsumowanie Choć wydawać by się mogło, że zwiększenie liczby granul, a tym samym liczby grupowanych danych, powinno poprawić jakość grupowania, wyniki klasteryzacji algorytmem GRAGGLO są lepsze, gdy liczba granul jest mniejsza – 50 lub 100. Gdy klastry mają kulisty kształt ('blobs', 'laguna'), granule wyznaczane są w ich środku, a im jest ich więcej, tym bardziej zagęszczone są w środku klastrów, co może powodować błędy obliczania przynależności danych bardziej oddalonych od tego centrum. Dodatkowo, w przypadku badanych

Table 19: Dokładność grupowania hierarchicznego danych w zależności od liczby granul i łączności

	dokładność	
liczba granul	SINGLELINKAGE	COMPLETELINKAGE
50	0.9792	0.6768
100	0.9872	0.6606
200	0.9853	0.6317

Table 20: Trafność grupowania hierarchicznego danych w zależności od liczby granul i łączności

	trafność	
liczba granul	SINGLELINKAGE	COMPLETELINKAGE
50	0.9639	0.6270
100	0.9700	0.6161
200	0.9352	0.5937

Table 21: Precyzja grupowania hierarchicznego danych w zależności od liczby granul i łączności

	precyzja	
liczba granul	SINGLELINKAGE	COMPLETELINKAGE
50	0.9658	0.6062
100	0.9698	0.5859
200	0.9389	0.5460

Table 22: Dokładność grupowania hierarchicznego danych w zależności od typu relacji i łączności

	dokładność	
typ relacji	SINGLELINKAGE	COMPLETELINKAGE
trójkątna	0.9952	0.6513
wykładnicza	0.9780	0.6570
gaussowska	0.9785	0.6608

Table 23: Trafność grupowania hierarchicznego danych w zależności od typu relacji i łączności

	trafność	
typ relacji	SINGLELINKAGE	COMPLETELINKAGE
trójkątna	0.9776	0.6123
wykładnicza	0.9476	0.6117
gaussowska	0.9438	0.6130

danych, dla większej liczby granul bardziej prawdopodobne było błędne wyznaczenie granul przez algorytm FCM.

Typu liczb rozmytych Średnią dokładność grupowania danych w zależności od typu relacji między granulami dla różnych łączności przedstawiono w tabeli 22, trafność – w tabeli 23, a precyzję – w tabeli 24. Wyniki w zależności od typu relacji są zbliżone, jednak najlepszą dokładność, trafność i precyzję odnotowano dla relacji trójkątnej i łączności SINGLELINKAGE. Wiąże się to ze skończonym nośnikiem liczb rozmytych. Jako że większość kształtów danych zawiera dysproporcje w liczebności, średnie wartości trafności i precyzji są niższe od odpowiadającym im wartościom dokładności.

Wyniki dla pomiarów, gdzie przynależność punktów do granul określono na podstawie relacji $*$-podobieństwa przedstawiono w tabelach 25, 26 i 27. Dla relacji wykładniczej i gaussowskiej nie występuje szum, dlatego wyniki nie różnią się, gdy zostanie uwzględniony. Gdy szumu nie ma, lub nie jest uwzględniony, wartości metryk są bardzo zbliżone do metryk dla przynależności określonej na podstawie macierzy $\boldsymbol{\mu}$.

W przypadku relacji trójkątnej, szum stanowi przeciętnie około 47% danych – wyniki, gdy zostaje on uwzględniony, są niemal o połowę niższe.

Table 24: Precyzja grupowania hierarchicznego danych w zależności od typu relacji i łączności

	precyzja	
typ relacji	SINGLELINKAGE	COMPLETELINKAGE
trójkątna	0.9762	0.5784
wykładnicza	0.9517	0.5790
gaussowska	0.9466	0.5807

Table 25: Dokładność grupowania hierarchicznego danych w zależności od typu relacji i łączności, gdy przynależność określana jest na podstawie relacji $*$-podobieństwa

	dokładność			
	bez szumu		z szumem	
typ relacji	SINGLELINKAGE	COMPLETELINKAGE	SINGLELINKAGE	COMPLETELINKAGE
trójkątna	0.9965	0.6265	0.4669	0.2999
wykładnicza	0.9779	0.6570	0.9779	0.6570
gaussowska	0.9783	0.6607	0.9783	0.6607

Table 26: Trafność grupowania hierarchicznego danych w zależności od typu relacji i łączności, gdy przynależność określana jest na podstawie relacji $*$-podobieństwa

	trafność			
	bez szumu		z szumem	
typ relacji	SINGLELINKAGE	COMPLETELINKAGE	SINGLELINKAGE	COMPLETELINKAGE
trójkątna	0.9671	0.5907	0.4461	0.2818
wykładnicza	0.9465	0.6092	0.9465	0.6092
gaussowska	0.9426	0.6116	0.9426	0.6116

Table 27: Precyzja grupowania hierarchicznego danych w zależności od typu relacji i łączności, gdy przynależność określana jest na podstawie relacji $*$-podobieństwa

	precyzja			
	bez szumu		z szumem	
typ relacji	SINGLELINKAGE	COMPLETELINKAGE	SINGLELINKAGE	COMPLETELINKAGE
trójkątna	0.9659	0.5635	0.4457	0.2721
wykładnicza	0.9507	0.5789	0.9507	0.5789
gaussowska	0.9454	0.5804	0.9454	0.5804

(a) SINGLELINKAGE (b) COMPLETELINKAGE

Figure 35: Zależność dokładności, trafności i precyzji grupowania od wartości parametru ξ.

Podsumowanie Najwyższe wartość metryk jakości przy grupowaniu algorytmem SINGLELINKAGE uzyskuje się dla relacji trójkątnej, choć relacja wykładnicza i gaussowska również ma wysokie wyniki. Wybór typu relacji ma za to duże znaczenie, gdy na jej podstawie określa się przynależność punktów do granul – relacja trójkątna powoduje wtedy, że istotna część danych oryginalnych może nie zostać w ogóle przypisana do żadnej grupy. Dla danych, które zostały zaklasyfikowane, wartości metryk są za to wyższe, niż dla pozostałych relacji, dla których wszystkie dane są zgrupowane.

Parametr ξ Średnią dokładność, trafność i precyzję grupowania danych w zależności od parametru ξ przedstawiono na rys. 35.

Dla SINGLELINKAGE, najwyższą dokładność i precyzję – odpowiednio 99.7745% i 98.1348% – osiągnięto dla $\xi = 0.05$, najwyższą trafność – 98.3582% – dla $\xi = 0.1$. Wartości metryk spadają wraz ze wzrostem wartości parametru. Powyżej $\xi = 0.8$ następuje gwałtowne załamanie na wykresie wyników.

Zależność dokładności jest przeciwna dla COMPLETELINKAGE: najlepsze wyniki – 67.1215% – osiągnięto dla $\xi = 0.9$. Trafność i precyzja nie wpisują się w tą zależność – ich wartości wahają się odpowiednio około 60-61% i 58%. Najwyższa wartość trafności wynosi 61.7007% dla $\xi = 0.65$, a precyzji – 58.4605% dla $\xi = 0.7$.

Podsumowanie Eksperymenty sugerują, że dla łączności SINGLELINKAGE najlepiej używać małych wartości parametru $\xi \approx 0.1$, a dla COMPLETELINKAGE –

dużych $\xi \approx 0.9$.

Kombinacja parametrów Wyniki pomiarów przeanalizowano też biorąc pod uwagę złożenie wpływu różnych parametrów – kształtu, liczby granul, typu relacji, parametru ξ – przy ich uśrednieniu dla wszystkich liczebności danych oryginalnych.

W tej sekcji zamieszczono wykresy złożonych zależności dla miary dokładności. W większości przypadków, trafność i precyzja posiadają na tyle podobne zależności, że nie jest konieczne dodatkowe opisywanie ich – wnioski byłyby identyczne. Wszystkie wykresy zamieszczono w katalogu `img/complex`, pogrupowane w podkatalogach według kształtu danych ('blobs', 'circles', 'corners', 'crescents', 'laguna' i 'spheres'). Nazwa każdego pliku wskazuje na metrykę, łączność i sposób określenia przynależności danych do granul:

1. metryka: dokładność (`acc`), trafność (`rec`), precyzja (`prec`),
2. łączność: SINGLELINKAGE (`sl`), COMPLETELINKAGE (`cl`),
3. przynależność danych do granul: na podstawie macierzy przynależności $\boldsymbol{\mu}$ (brak oznaczenia), na podstawie relacji $*$-podobieństwa (`shape_dep`).

'blobs' Jak pokazuje rys. 36, dla dokładności grupowania danych 'blobs' zdecydowanie największe znaczenie ma liczba granul. Najlepsze wyniki – około 100% niezależnie od innych parametrów – uzyskiwane są dla 50 granul. Dla takiej liczebności algorytm rozmytych c-średnich zazwyczaj tworzył granule we wszystkich trzech oczekiwanych klastrach.

Mniejsze znaczenie ma parametr ξ, jednak da się zauważyć spadek dokładności dla większych jego wartości dla SINGLELINKAGE i wzrost dokładności dla COMPLETELINKAGE, co jest zgodne z ogólną tendencją, przedstawioną na rys. 35. Typ relacji nie ma wyraźnego wpływu na jakość grupowania.

Jeżeli granule znalezione przez algorytm rozmytych c-średnich znajdą się we wszystkich trzech klastrach, to grupowanie FUZZYAGGLO przebiega poprawnie. W przeciwnym razie w centralnym klastrze, pojedyncze granule uznane zostaną za należące do innego klastra, jako że algorytm musi wyznaczyć trzy grupy (rys. 20). Parametr ξ i typ relacji mogą wtedy wpłynąć na to, ile punktów danych oryginalnych znajdzie się w tych nieprawidłowych klastrach, co prowadzi do fluktuacji dokładności.

'circles' Wykresy dla danych 'circles' (rys. 37) uwydatniają wpływ łączności na jakość grupowania.

(a) SingleLinkage

(b) CompleteLinkage

Figure 36: Zależność dokładności grupowania danych ‘blobs’ od liczby granul, typu relacji i wartości parametru ξ.

(a) SingleLinkage

(b) CompleteLinkage

Figure 37: Zależność dokładności grupowania danych ‘circles’ od liczby granul, typu relacji i wartości parametru ξ.

W przypadku łączności SingleLinkage, dokładność jest niemal zawsze równa 100%. Spada ona dla wysokich (≥ 0.9) wartości parametru ξ. Najniższe wartości są osiągane dla relacji wykładniczej, wyższe dla gaussowskiej. Dla relacji trójkątnej, wyniki pozostają na poziomie 100% nawet dla $\xi = 0.95$.

Algorytm dla łączności CompleteLinkage nie odnajduje poprawnie klastrów, co widać na rys. 29. Badane parametry nie mają znacznego wpływu na jakość.

‘corners’ Podobnie jak w przypadku ‘circles’, grupowanie danych ‘corners’ przy łączności CompleteLinkage nie zwraca poprawnych wyników. Rys. 38 pokazuje, że przez to pozostałe parametry nie mają zauważalnego wpływu na jakość grupowania.

Dla łączności SingleLinkage największe znaczenie ma parametr ξ. Dokładność zmniejsza się wraz ze wzrostem wartości tego parametru. Ponadto znaczenie ma też liczba granul i typ relacji. Dokładność wcześniej zaczyna spadać dla mniejszej liczebności granul, najpierw dla relacji gaussowskiej, potem wykładniczej, na końcu trójkątnej. Dla relacji trójkątnej i większych liczebności granul – 100 i 200 – średnia dokładność zostaje na poziomie 100% nawet dla $\xi = 0.95$.

‘crescents’ Wnioski dla danych ‘crescents’ są bardzo podobne do wniosków dla danych ‘circles’. Jak pokazuje rys. 39, dla łączności SingleLinkage średnia

(a) SingleLinkage

(b) CompleteLinkage

Figure 38: Zależność dokładności grupowania danych ‘corners’ od liczby granul, typu relacji i wartości parametru ξ.

(a) SingleLinkage

(b) CompleteLinkage

Figure 39: Zależność dokładności grupowania danych ‘crescents’ od liczby granul, typu relacji i wartości parametru ξ.

(a) SingleLinkage

(b) CompleteLinkage

Figure 40: Zależność dokładności grupowania danych ‘laguna’ od liczby granul, typu relacji i wartości parametru ξ.

dokładność spada poniżej 100% dopiero dla wysokich (≥ 0.9) wartości ξ, najbardziej dla relacji wykładniczej. W przypadku CompleteLinkage, również można zauważyć spadek dokładności dla tej relacji i wysokiego ξ, jednak nie ma jasnego wpływu badanych parametrów na jakość grupowania.

‘laguna’ Przy łączności SingleLinkage dokładność spada wraz ze wzrostem wartości ξ – rys. 40. Najszybciej jakość zmniejsza się dla dużej liczebności granul – 200. Ma to związek z nieprawidłową przynależnością punktów do granul na zewnątrz dwóch wewnętrznych klastrów, jak widać na rys. 26. Tak jak w przypadku innych kształtów danych, najniższe wyniki są uzyskiwane dla relacji wykładniczej.

Obliczanie przynależności może też wpływać na wyniki dla CompleteLinkage. Dla tej łączności znowu nie można jasno określić wpływu parametrów na dokładność grupowania, natomiast zauważyć można, że dla większych liczebności granul wyniki są bardziej zróżnicowane, natomiast dla 50 granul są one bliższe średniej.

‘spheres’ Wyniki dla danych ‘spheres’ (rys. 41) przedstawiają inne zależności od wcześniej opisanych danych.

Przy łączności SingleLinkage, dokładność zmniejsza się wraz ze wzrostem parametru ξ, jednak najgorsze wyniki otrzymywane są dla najmniejszej liczebności granul – 50. Gdy jest ich mniej, granule na zewnętrznej sferze mogą być znacznie oddalone od siebie nawzajem, przez co sfera wewnętrzna łączona jest z zewnętrzną, a pojedyncza granula ze sfery zewnętrznej zostaje uznana za osobny klaster (rys. 27).

Dla CompleteLinkage można zauważyć wzrost średniej dokładności grupowania wraz ze wzrostem parametru ξ, szczególnie dla 50 granul.

(a) SingleLinkage

(b) CompleteLinkage

Figure 41: Zależność dokładności grupowania danych ‘spheres’ od liczby granul, typu relacji i wartości parametru ξ.

Podsumowanie Łączność SingleLinkage jest znacznie lepszym wyborem podczas grupowania badanych danych niż CompleteLinkage. Dla tej łączności, niezależnie od kształtu danych, najlepsze wyniki uzyskuje się dla niskich wartości parametru progowego ξ. Najlepiej wykorzystywać też relację trójkątną – z nielicznymi wyjątkami przy danych ‘blobs’, wyniki są najwyższe dla tego kształtu granul, przy zwiększaniu parametru ξ jakość najpóźniej spada dla tej relacji.

Wnioski dotyczące optymalnej liczby granul zależą od kształtu danych. Gdy klastry danych układają się w kształt kulisty – ‘blobs’, ‘laguna’ – lepsze są małe liczby granul ($G = 50$), co poprawia jakość klasteryzacji algorytmem rozmytych c-średnich. Dla pozostałych badanych kształtów lepiej wybierać większe liczby, dzięki czemu granule rozkładają się bardziej równomiernie.

Przy grupowaniu danych algorytmem o łączności CompleteLinkage, w większości przypadków nie widać jasnego wpływu parametrów na wyniki, za wyjątkiem grupowania danych ‘blobs’. Dla tych danych najlepsze wyniki uzyskiwane są dla małej liczby granul ($G = 50$), wysokiej wartości ξ oraz relacji wykładniczej.

Dane ‘brightkite’ i ‘gowalla’ Ze względu na brak etykiet danych ‘brightkite’ i ‘gowalla’, jakość grupowania można ocenić wizualnie, na podstawie wykresów – odpowiednio rys. 42 i 43.

Podczas grupowania ostrego algorytmem `AgglomerativeClustering` wykorzystano losową próbkę $500\,000$ współrzędnych, jako że algorytm ten nie nadaje się do grupowania bardzo dużych ilości danych. Jako parametry grupowania GRAGGLO wybrano na podstawie wyników przedstawionych na stronie 28 i dalszych, relację trójkątną, 100 granul i $\xi = 0.1$. W obydwu przypadkach jako docelową liczbę klastrów wybrano 6 – jeden klaster to jeden kontynent.

Grupowanie ostre algorytmem `AgglomerativeClustering` nie odnajduje oczekiwanych klastrów. W przypadku danych ‘brightkite’, wszystkie kontynenty zostały połączone w jeden klaster, zaś elementy odstające – wyspy, ocean, błędne współrzędne – to pozostałe 5 klastrów. W przypadku danych ‘gowalla’ grupowanie jest lepsze – Ameryki stanowią osobny klaster niż reszta kontynentów – jednak wciąż 4 klastry odpowiadają wyłącznie danym odstającym.

Wyniki grupowania algorytmem GRAGGLO są dużo bliższe oczekiwaniom. Wstępna granulacja FCM pozwoliła na eliminację szumów (w tym błędnych współrzędnych), na które wrażliwy jest algorytm grupowania hierarchicznego. Zauważyć

(a) Grupowanie ostre próbki losowej $500\,000$ punktów

(b) Grupowanie algorytmem GRAGGLO całości danych

Figure 42: Wyniki grupowania hierarchicznego algorytmem ostrym `AgglomerativeClustering` i algorytmem GRAGGLO danych 'brightkite'

(a) Grupowanie ostre próbki losowej $500\,000$ punktów

(b) Grupowanie algorytmem GRAGGLO całości danych

Figure 43: Wyniki grupowania hierarchicznego algorytmem ostrym `AgglomerativeClustering` i algorytmem GRAGGLO danych 'gowalla'

można, że duży wpływ na rozmieszczenie granul ma zagęszczenie danych – najwięcej granul powstało we współrzędnych odpowiadających Europie i Ameryce Północnej. Zarówno w przypadku danych 'brightkite', jak i 'gowalla', Europa i Azja zostały uznane za jeden klaster.

6 Podsumowanie

Do analizy danych powszechnie wykorzystuje się algorytmy grupowania. Polegają one na wyznaczeniu takich grup danych, aby wszystkie dane przypisane do tej samej grupy były do siebie jak najbardziej podobne pod względem wybranej metryki, a dane przypisane do różnych grup – jak najbardziej niepodobne. Algorytmy takie mogą być na przykład oparte na minimalizacji pewnej funkcji kryterialnej bądź wykorzystywać zagęszczenie danych w pewnym obszarze do wyznaczenia klastrów. W pracy skupiono się na aglomeracyjnym algorytmie hierarchicznym, który polega na tworzeniu sekwencji zagnieżdżonych podziałów danych poprzez łączenie mniejszych klastrów w większe. Wadą tego algorytmu jest jego duża złożoność czasowa: sześcienna dla implementacji ogólnej. Choć istnieją specjalizowane implementacje o niższych złożonościach (kwadratowych), to pozostają one na tyle wysokie, że algorytm jest problematyczny do wykorzystania dla dużych liczebności danych.

W pracy do przyspieszenia działania aglomeracyjnego algorytmu hierarchicznego wykorzystano obliczenia granularne. Jest to dziedzina, która również stosowana jest do analizy danych. Polega na analizowaniu ich na sposób podobny do ludzkiego postrzegania rzeczywistości. Związanym z nią pojęciem jest granula informacyjna, będąca encją o pewnej semantyce. W pracy granule realizowane są przy pomocy liczb rozmytych, czyli takich liczb, które modelują pewne ostre wartości liczbowe w jakimś przybliżeniu.

W pracy zaproponowany został autorski algorytm GRAGGLO, który wykonuje grupowanie hierarchiczne na granulach informacyjnych, zamiast na oryginalnych liczbach ostrych. Krokiem przygotowawczym algorytmu jest zgrupowanie danych wejściowych (współrzędnych punktów) w granule. Wykorzystano do tego algorytm rozmytych c-średnich FCM, który ma mniejszą – liniową – złożoność czasową niż algorytm hierarchiczny. Algorytm ten grupuje dane w klastry rozmyte, czyli takie, do których każdy punkt należy w pewnym stopniu, określonym jako liczba z przedziału $[0, 1]$. Następnie klastry te przekształcane są w granule: wielowymiarowe rozszerzalne liczby rozmyte, gdzie dla każdego wymiaru wyróżnić można środek i rozmycie granuli (ziaren informacyjnych). Wykorzystano trzy kształty ziaren: trójkątne, wykładnicze i gaussowskie, definiowane przez odpowiednie rozmyte relacje $*$-podobieństwa. Właściwe aglomeracyjne grupowanie hierarchiczne algorytmu GRAGGLO wykorzystuje ogólną implementację o złożoności sześciennej dostosowaną do operowania na granulach. Modyfikacje algorytmu w stosunku do standardowej, ostrej wersji, dotyczą operacji arytmetycznych i logicznych. Odległości euklidesowe między granulami również mają postać liczb rozmytych, które są następnie porównywane. Jako że wynikiem porównań liczb rozmytych jest wartość z przedziału $[0, 1]$, wprowadzony został dodatkowy parametr progowy ξ, określający, dla jakiej wartości operacja logiczna będzie ewaluowana jako prawda. Zaimplementowano dwie łączności grupowania hierarchicznego: SINGLELINKAGE oraz COMPLETELINKAGE.

W pracy przebadano własności zaproponowanego algorytmu – jego złożoność czasową oraz jakość grupowania. Porównano czas wykonania algorytmu GRAGGLO z czasem wykonania ostrego algorytmu hierarchicznego o analogicznej implementacji dla różnych liczebności danych. Odnotowano znaczne zmniejszenie czasu wykonania – przykładowo, dla 5000 danych algorytm GRAGGLO wykonywał się 1-4 sekundy (w zależności od liczby granul), a algorytm ostry – około 100 sekund. Możliwe było też wykonanie algorytmu na bardzo dużych danych – kilku milionach punktów – w czasie rzędu kilkudziesięciu minut. Ponadto algorytm jest też szybszy od optymalnego algorytmu bibliotecznego powyżej pewnej liczby danych. Jego złożoność czasowa jest liniowa w zależności od liczby danych wejściowych – choć właściwe grupowanie hierarchiczne nadal ma złożoność sześcienną, to wykorzystuje ono granule informacyjne, których jest znacznie mniej niż danych wejściowych. Udało się więc osiągnąć główny cel pracy – znacznie zmniejszono czas wykonywania algorytmu hierarchicznego dla dużych liczebności danych. W zamian za przyspieszenie algorytmu dopuszczono pogorszenie jakości grupowania. Pomiary dokładności, trafności i precyzji algorytmu GRAGGLO pokazały jednak, że w większości przypadków jego wyniki nie odbiegają od wyników algorytmu ostrego, a nawet mogą być lepsze. Algorytm badano dla różnych kombinacji parametrów: liczby granul, kształtu granul i parametru progowego ξ, dla obydwu zaimplementowanych łączności. Dla wszystkich parametrów, średnie war-

tości metryk jakości są zbliżone do wyników tych samych metryk dla algorytmu ostrego. Ponadto w większości przypadków można znaleźć taką kombinację tych parametrów, dla których wyniki są lepsze niż dla algorytmu ostrego. Analiza grupowania GRAGGLO pokazuje również, że granulacja danych wejściowych algorytmem rozmytych c-średnich pozwala na wyeliminowanie z danych elementów odstających, na które bardzo wrażliwy jest algorytm hierarchiczny.

Implementację algorytmu GRAGGLO oraz skrypty wykorzystane do badań jego działania zamieszczono w repozytorium pod adresem `https://github.com/ZofiaLor/magister-granule`.

Algorytm GRAGGLO może w prosty sposób zostać rozszerzony o inne metryki odległości. Dzięki zastosowaniu ogólnej implementacji, łatwo można też dodać inne opcje łączności, takie jak łączność średnia lub Warda. Ponadto implementację granul rozszerzyć można o inne kształty ziaren poprzez zastosowanie innych rozmytych relacji $*$-podobieństwa.

References

[1] Lasso selector — matplotlib 3.10.9 documentation, 2026.

[2] Mihael Ankerst, Markus M. Breunig, Hans-Peter Kriegel, and Jörg Sander. Optics: ordering points to identify the clustering structure. In *Proceedings on 1999 ACM SIGMOD international conference on management of data*, pages 49–60, 1999.

[3] James C. Bezdek. Cluster validity with fuzzy sets. *Cybernetics and Systems*, 3(3):58–73, 1973.

[4] Aritra Bhowmick, Mert Kosan, Zexi bit Huang, Ambuj Singh, and Sourav Medya. DGCLUSTER: A neural framework for attributed graph clustering via modularity maximization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 11069–11077, March 2024.

[5] D. Defays. An efficient algorithm for a complete link method. *The Computer Journal*, 20(4):364–366, 01 1977.

[6] J. Leskovec E. Cho, S. A. Myers. Snap: Network datasets: Brightkite, 2011.

[7] J. Leskovec E. Cho, S. A. Myers. Snap: Network datasets: Gowalla, 2011.

[8] Martin Ester, Hans-Peter Kriegel, Jörg Sander, and Xiaowei Xu. A density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, pages 226–231, 1996.

[9] Kazimierz Florek, Józef Łukaszewicz, Julian Perkal, Hugo Steinhaus, and Stefan Zubrzycki. Sur la liaison et la division des points d'un ensemble fini. *Colloquium Mathematicum*, 2(3-4):282–285, 1951.

[10] Soumi Ghosh and Sanjay Kumar. Comparative analysis of k-means and fuzzy c-means algorithms. *International Journal of Advanced Computer Science and Applications*, 4:35–39, 2013.

[11] Seyed Emadedin Hashemi, Fatemeh Gholian-Jouybari, and Mostafa Hajiaghaei-Keshteli. A fuzzy c-means algorithm for optimizing data clustering. *Expert Systems with Applications*, 227:120377, 2023.

[12] Michal Holčapek and Martin Štěpnička. Arithmetics of extensional fuzzy numbers - part I: Introduction. In *2012 IEEE International Conference on Fuzzy Systems*, pages 1–8, 2012.

[13] Michal Holčapek and Martin Štěpnička. Arithmetics of extensional fuzzy numbers - part II: Algebraic framework. In *2012 IEEE International Conference on Fuzzy Systems*, pages 1–8, 2012.

[14] Michal Holčapek and Martin Štěpnička. MI-algebras: A new framework for arithmetics of (extensional) fuzzy numbers. *Fuzzy Sets and Systems*, 257:102–131, 2014. Special Issue on Fuzzy Numbers and Their Applications.

[15] Jacek M. Leski. Fuzzy c-ordered-means clustering. *Fuzzy Sets and Systems*, 286:114–133, 2016.

[16] Zheng Liu, Shiluo Huang, Wei Jin, and Ying Mu. Local kernels based graph learning for multiple kernel clustering. *Pattern Recognition*, 150:110300, 2024.

[17] Louis L. McQuitty. Elementary linkage analysis for isolating orthogonal and oblique types and diversified groups. *Educational and Psychological Measurement*, 17(2):207–229, 1957.

[18] Feiping Nie, Cheng Liu, Rong Wang, and Xuelong Li. A novel and effective method to directly solve spectral clustering. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 46(12):10863–10875, 2024.

[19] Adam Pedrycz, Kaoru Hirota, Witold Pedrycz, and Fangyan Dong. Granular representation and granular computing with fuzzy sets. *Fuzzy Sets and Systems*, 203:17–32, 2012.

[20] Andrzej Piegat and Marcin Pluciński. Computing with words with the use of inverse RDM models of membership functions. *International Journal of Applied Mathematics and Computer Science*, 25(3):675–688, 2015.

[21] Piotr Pieta and Tomasz Szmuc. Applications of rough sets in big data analysis: An overview. *International Journal of Applied Mathematics and Computer Science*, 31(4):659–683, 2021.

[22] Wojciech Ptas and Krzysztof Siminski. The art of saying 'I don't know': Learned deferment in neuro-fuzzy three-way decision cascade classifiers. *Axioms*, 15(5), 2026.

[23] Yazhou Ren, Jie Pu, Chao Cui, Yu Zheng, Xinyue Chen, Xiaorong Pu, and Lifang He. Dynamic weighted graph fusion for deep multi-view clustering. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*, IJCAI '24, pages 4835–4843, August 2024.

[24] R. Sibson. Slink: An optimally efficient algorithm for the single-link cluster method. *The Computer Journal*, 16(1):30–34, 01 1973.

[25] Krzysztof Siminski. Clustering with missing values. *Fundamenta Informaticae*, 123(3):331–350, 2013.

[26] Krzysztof Siminski. Neuro-fuzzy system with weighted attributes. *Soft Computing*, 18(2):285–297, 2014.

[27] Krzysztof Siminski. Rough fuzzy subspace clustering for data with missing values. *Computing & Informatics*, 33(1):131–153, 2014.

[28] Krzysztof Siminski. GrFCM - granular clustering of granular data. In Aleksandra Gruca, Tadeusz Czachórski, Sebastian Deorowicz, Katarzyna Harezlak, and Agnieszka Piotrowska, editors, *Man-Machine Interactions 6*, pages 111–121, Cham, 2020. Springer International Publishing.

[29] Krzysztof Siminski. GrNFS - granular neuro-fuzzy system for regression in large volume data. *International Journal of Applied Mathematics and Computer Science*, 31(3):445–459, 2021.

[30] Krzysztof Siminski. FuBiNFS – fuzzy biclustering neuro-fuzzy system. *Fuzzy Sets and Systems*, 438:84–106, 2022.

[31] Krzysztof Siminski. 3WDNFS – three-way decision neuro-fuzzy system for classification. *Fuzzy Sets and Systems*, 466:108432, 2023.

[32] Krzysztof Siminski. Arithmetic with extensional fuzzy numbers. *Fuzzy Sets and Systems*, 544:110067, 2026.

[33] Krzysztof Siminski and Wojciech Ptas. Towards parameterless three-way decision cascade of neuro-fuzzy classifiers. In *Proceedings of the 7th Polish Conference on Artificial Intelligence, PP-RAI 2026*, 2026.

[34] Andrzej Skowron, Andrzej Jankowski, and Soma Dutta. Interactive granular computing. *Granular Computing*, 1:95–113, 2016.

[35] Peter H. A. Sneath. The application of computers to taxonomy. *Journal of General Microbiology*, 17(1):184–200, 1957.

[36] Robert Reuven Sokal and Charles Duncan Michener. A statistical method for evaluating systematic relationships. *University of Kansas Science Bulletin*, 38(22):1409–1438, 1958.

[37] Thorvald Sørensen. A method of establishing groups of equal amplitude in plant sociology based on similarity of species content and its application to analyses of the vegetation on danish commons. *Biologiske Skrifter / Kongelige Danske Videnskabernes Selskab*, 5:1–34, 1948.

[38] Dawid Suchy and Krzysztof Siminski. GrDBSCAN: A granular density-based clustering algorithm. *International Journal of Applied Mathematics and Computer Science*, 33(2):297–312, 2023.

[39] Jun Tang, Min Zhang, Ruipeng Ma, and Jun Xiao. A density-driven graph-based clustering model with adaptive outlier recognition. *Information Processing & Management*, 63(2):104436, 2025.

[40] Xinxin Wang, Yongshan Zhang, and Yicong Zhou. Bidirectional probabilistic multi-graph learning and decomposition for multi-view clustering. *IEEE Transactions on Image Processing*, 34:3609–3621, 2025.

[41] J. H. Ward, Jr. Hierarchical grouping to optimize an objective function. *Journal of the American Statistical Association*, 58:236–244, 1963.

[42] W.E. Wright. Gravitational clustering. *Pattern Recognition*, 9(3):151–166, 1977.

[43] Sisi Xia, Lin Chen, Siya Liu, and Haoran Yang. A new method for decision making problems with redundant and incomplete information based on incomplete soft sets: from crisp to fuzzy. *International Journal of Applied Mathematics and Computer Science*, 32(4):657–669, 2022.

[44] Dongkuan Xu and Ying jie Tian. A comprehensive survey of clustering algorithms. *Annals of Data Science*, 2:165–193, 2015.

[45] Yiyu Yao. Three-way decision: An interpretation of rules in rough set theory. In Peng Wen, Yuefeng Li, Lech Polkowski, Yiyu Yao, Shusaku Tsumoto, and Guoyin Wang, editors, *Rough Sets and Knowledge Technology*, pages 642–649, Berlin, Heidelberg, 2009. Springer Berlin Heidelberg.

[46] Yiyu Yao. The superiority of three-way decisions in probabilistic rough set models. *Information Sciences*, 181(6):1080–1096, 2011.

[47] Yiyu Yao. A triarchic theory of granular computing. *Granular Computing*, 1:145–157, 2016.

[48] Lotfi A. Zadeh. Fuzzy sets. *Information and Control*, 8:338–353, 1965.

[49] Lotfi A. Zadeh. Fuzzy sets and information granularity. In N. Gupta, R. Ragade, and R. Yager, editors, *Advances in Fuzzy Set Theory and Applications*, pages 3–18. North-Holland Publishing Co., 1979.

[50] Lotfi A. Zadeh. Toward a theory of fuzzy information granulation and its centrality in human reasoning and fuzzy logic. *Fuzzy Sets and Systems*, 90(2):111–127, 1997.

[51] Lotfi A. Zadeh. From computing with numbers to computing with words—from manipulation of measurements to manipulation of perceptions. *International Journal of Applied Mathematics and Computer Science*, 12(3):307–324, 2002.

Algorithm 6 Wyznaczenie przynależności punktu do granuli na podstawie relacji $*$-podobieństwa S

Require: $\mathbb{X}$: dane oryginalne
Require: $\mathbb{G}$: granule wyznaczone przez algorytm GRAGGLO
Require: $\mathbf{e}_g$: wektor o długości G etykiet wyznaczonych dla wszystkich granul
Require: r: typ relacji (trójkątna, wykładnicza, gaussowska)

procedure CALCULATESIMILARITYMEMBERSHIP($\mathbb{X}, \mathbb{G}, \mathbf{e}, r$)
    $\mathbf{e}_x \leftarrow \varnothing$ ▷ etykiety punktów
    $szum \leftarrow 0$ ▷ liczba punktów nieprzynależących do żadnej granuli
    for x in X **do**
        $e \leftarrow 0$
        $\mu_{max} \leftarrow 0$
        for $i = 1$ to G **do**
            $\mu \leftarrow (\mathbb{G}[i][1] =_S x[1])$
            for $j = 2$ to liczba wymiarów x **do**
                $\mu \leftarrow \mu \star (\mathbb{G}[i][j] =_S x[j])$ ▷ $\star$: t-norma
            end for
            if $\mu > \mu_{max}$ **then**
                $\mu_{max} \leftarrow \mu$
                $e \leftarrow i$
            end if
        end for
        if $\mu_{max} > 0$ **then**
            dodaj i do $\mathbf{e}_x$
        else ▷ punkt nie przynależy do żadnej granuli – jest szumem
            dodaj -1 do $\mathbf{e}_x$
            $szum \leftarrow szum + 1$
        end if
    end for
    return $\mathbf{e}_x, szum$
end procedure

Algorithm 7 Wyznaczenie macierzy $\mathbf{L}$

Require: $\boldsymbol{\mu}$: macierz $X \times X$ przynależności danych do klastrów
Require: $\mathbf{e}_r$: wektor o długości X etykiet wyznaczonych dla wszystkich punktów przez algorytm ostry
Require: $\mathbf{e}_g$: wektor o długości C etykiet wyznaczonych dla wszystkich granul przez algorytm GRAGGLO

procedure CALCULATELMATRIX($\boldsymbol{\mu}, \mathbf{e}_r, \mathbf{e}_g$)
    $\mathbf{L} \leftarrow$ macierz zerowa $C \times C$
    for $i = 1$ to X **do**
        $\mu \leftarrow$ ARGMAX($\boldsymbol{\mu}[i]$) ▷ Indeks identyfikujący granulę, dla której przynależność punktu i-tego jest największa
        inkrementacja $\mathbf{L}[\mathbf{e}_r[i]][\mathbf{e}_g[\mu]]$
    end for
    return $\mathbf{L}$
end procedure

Algorithm 8 Permutacja macierzy $\mathbf{L}$

Require: $\mathbf{L}$: macierz $C \times C$, wyznaczona przez algorytm 7

procedure PERMUTELMATRIX($\mathbf{L}$)
    for $i = 1$ to $C - 1$ **do**
        $(w, k) \leftarrow \arg\max$(**L**[i:][i:]) ▷ Indeksy wiersza i kolumny, w których znajduje się maksymalna wartość podmacierzy $\mathbf{L}[i:][i:]$
        $w \leftarrow w + i$
        $k \leftarrow k + i$
        zamiana kolejnością wiersza i-tego i w-tego macierzy $\mathbf{L}$
        zamiana kolejnością kolumny i-tej i k-tej macierzy $\mathbf{L}$
    end for
    return $\mathbf{L}$
end procedure

Algorithm 9 Wyznaczenie dokładności, trafności i precyzji

Require: $\mathbf{L}$: macierz $C \times C$, otrzymana na podstawie algorytmu 8
Require: X: liczba danych
procedure CALCULATEQUALITYMETRICS($\mathbf{L}, X$)
    $diag \leftarrow 0$ ▷ Suma wartości na przekątnej macierzy $\mathbf{L}$
    $R \leftarrow 0$
    $P \leftarrow 0$
    $suma_R \leftarrow$ sumy wartości dla każdego wiersza macierzy $\mathbf{L}$
    $suma_P \leftarrow$ sumy wartości dla każdej kolumny macierzy $\mathbf{L}$
    for $i = 1$ to C **do**
        $diag \leftarrow diag + \mathbf{L}[i, i]$
        if $suma_R[i] > 0$ **then**
            $R \leftarrow R + \mathbf{L}[i, i]/suma_R[i]$
        end if
        if $suma_P[i] > 0$ **then**
            $P \leftarrow P + \mathbf{L}[i, i]/suma_P[i]$
        end if
    end for
    $A \leftarrow diag/X$
    $R \leftarrow R/C$
    $P \leftarrow P/C$
    return A, R, P
end procedure